

Intervals & Sweep Line

57. Insert Interval

To a senior engineer (L5) or staff engineer (L6) at Google, “Insert Interval” is a masterclass in **boundary condition management**. While a junior might try to find the insertion point and then call a separate “merge” function (which is inefficient), a senior engineer solves this in a single, clean pass.

They view the problem as a **three-stage stream processing** task: things that happen *before* the new interval, things that *overlap* with it, and things that happen *after*.

1. Problem Explanation

The Goal: You have a list of non-overlapping intervals, sorted by their start times (e.g., `[[1,3], [6,9]]`). You are given a `newInterval` (e.g., `[2,5]`). You must insert this new interval into the list and merge any overlaps so the final list remains sorted and non-overlapping.

What is an Interval? It is a range `[start, end]`. Two intervals overlap if one starts before the other ends. * `[1, 5]` and `[6, 8]` -> **No Overlap** (6 is after 5). * `[1, 5]` and `[4, 8]` -> **Overlap** (4 is before 5).

The Challenge: The `newInterval` could: 1. Fit perfectly in a gap. 2. Swallow up multiple existing intervals. 3. Partially overlap at the beginning or the end.

2. Solution Explanation

The Intuition (The “Why”): Instead of modifying the list in place (which is slow in many languages due to array shifting), we build a **new list** by categorizing every existing interval into one of three buckets based on its relationship with the `newInterval`.

The Three Stages (The “How”):

1. **Stage 1: The “Before” Bucket** Any interval that ends *before* the `newInterval` starts. These are unaffected. We just copy them over.
 - *Condition:* `interval.end < newInterval.start`
2. **Stage 2: The “Merge” Bucket** Any interval that overlaps with the `newInterval`. We don’t add them yet. Instead, we “absorb” them into the `newInterval` by stretching its boundaries.
 - *Condition:* `interval.start <= newInterval.end` (since we already cleared Stage 1).
 - *Action:* `newStart = min(newStart, interval.start), newEnd = max(newEnd, interval.end)`.

3. **Stage 3: The “After” Bucket** Once the merging is done, we add the final merged `newInterval`. Then, we simply copy over all remaining intervals that start *after* our merged interval ends.

Walkthrough with ASCII Visualization: Input: `intervals = [[1,2], [3,5], [6,7], [8,10], [12,16]]`, `newInterval = [4,8]`

Time Line: 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16
Existing: [1-2] [3---5] [6-7] [8---10] [12-----16]
New: [4-----8]

STAGE 1: Before (`interval.end < newInterval.start`)
Check [1,2]: `2 < 4`? YES.
Action: Add [1,2] to Result.
Result: [[1,2]]

STAGE 2: Overlap (`interval.start <= newInterval.end`)
Check [3,5]: `3 <= 8`? YES.
Action: Merge! `newInterval` becomes [`min(4,3)`, `max(8,5)`] = [3,8]

Check [6,7]: `6 <= 8`? YES.
Action: Merge! `newInterval` becomes [`min(3,6)`, `max(8,7)`] = [3,8]

Check [8,10]: `8 <= 8`? YES.
Action: Merge! `newInterval` becomes [`min(3,8)`, `max(8,10)`] = [3,10]

Check [12,16]: `12 <= 10`? NO.
Action: Overlap stage finished. Add merged interval [3,10] to Result.
Result: [[1,2], [3,10]]

STAGE 3: After (Everything else)
Check [12,16]: Remaining.
Action: Add to Result.

FINAL RESULT: [[1,2], [3,10], [12,16]]

3. Time and Space Complexity Analysis

TIME COMPLEXITY: $O(N)$

- - We use a single 'while' loop or three sequential loops.
- We visit each interval in the input list exactly once.

- Every operation (min, max, array push) is constant $O(1)$.
- Total time scales perfectly linearly with the number of intervals (N).

[Linear Traversal Chart]
 Intervals: | 1 | 2 | 3 | 4 | 5 | ... | N |
 Processing: ^ ^ ^ ^ ^
 Steps: $O(1)+O(1)+O(1)+O(1)+O(1)+\dots+O(1) = O(N)$

SPACE COMPLEXITY: $O(N)$

- We create a new list to store the results.
- In the worst case (no overlaps), the result list will have $N + 1$ intervals.
- Therefore, memory usage grows linearly with N .

[Memory Allocation]
 Input: [][][][] (Size N)
 Result: [][][][][] (Size $N+1$)

4. Solution Code

Python Snippet

```
def insert(intervals, newInterval):
    result = []
    i = 0
    n = len(intervals)

    # STAGE 1: Add all intervals ending before newInterval starts
    # These are completely to the left.
    while i < n and intervals[i][1] < newInterval[0]:
        result.append(intervals[i])
        i += 1

    # STAGE 2: Merge overlapping intervals
    # While the current interval starts before or when newInterval ends, they overlap.
    while i < n and intervals[i][0] <= newInterval[1]:
        # Expand newInterval to cover the current interval
        newInterval[0] = min(newInterval[0], intervals[i][0])
        newInterval[1] = max(newInterval[1], intervals[i][1])
        i += 1
```

```

# Add the final merged newInterval
result.append(newInterval)

# STAGE 3: Add all remaining intervals
# These are completely to the right.
while i < n:
    result.append(intervals[i])
    i += 1

return result

```

JavaScript Snippet

```

function insert(intervals, newInterval) {
    const result = [];
    let i = 0;
    const n = intervals.length;

    // 1. Process intervals that come strictly BEFORE the new one
    while (i < n && intervals[i][1] < newInterval[0]) {
        result.push(intervals[i]);
        i++;
    }

    // 2. Process OVERLAPPING intervals and merge them into newInterval
    while (i < n && intervals[i][0] <= newInterval[1]) {
        newInterval[0] = Math.min(newInterval[0], intervals[i][0]);
        newInterval[1] = Math.max(newInterval[1], intervals[i][1]);
        i++;
    }

    // Push the result of the merge
    result.push(newInterval);

    // 3. Process intervals that come strictly AFTER the new one
    while (i < n) {
        result.push(intervals[i]);
        i++;
    }

    return result;
}

```

Note 1: Terminology Addendum

Greedy/Linear Processing: * **What it is:** Making the locally optimal choice at each step (handling intervals in order) to reach a globally correct solution. * **Why it helps:** Because the input is already sorted, we don't need complex data structures like trees. We just "stream" the data through our logic.

Boundary Conditions: * **What it is:** The "edges" of your logic (e.g., empty lists, the very first element, the very last element). * **How it applies:** The three-stage approach naturally handles cases where `newInterval` goes at the very beginning or very end without needing special `if` statements.

Note 2: Real-World Interview Variations

Variation 1: "Meeting Rooms III" (Google/Meta) * **The Prompt:** You have a list of meeting times and a limited number of rooms. If a new meeting comes in and all rooms are full, delay it until a room is free. * **The L5 Solve:** This uses the same "overlap" logic but introduces a **Min-Heap (Priority Queue)**. You track when each room becomes free. When inserting a new "interval" (meeting), you check for overlaps with free time.

Variation 2: "Employee Free Time" (Bloomberg/Meta) * **The Prompt:** Given the schedules of several employees, find the time intervals where *everyone* is free. * **The L5 Solve:** You take all intervals from all employees and sort them (or use a heap). Then, you perform a "Merge Intervals" pass. The "gaps" *between* your merged intervals are the free time.

Variation 3: "Calendar Conflict Checker" (System Design Context) * **The Prompt:** Design a backend for Google Calendar that prevents users from double-booking. * **The L5 Solve:** At the database level, this is an interval insertion problem. For efficiency in a large database, you would use an **Interval Tree** or a **Segment Tree** so you can find overlaps in $O(\log N)$ time rather than $O(N)$. But for a single user's daily view, the linear $O(N)$ scan presented above is the production standard for speed.

56. Merge Intervals

This is a classic "sweep-line" or sorting-based problem. A Senior Engineer (L5/L6) at Google views this through the lens of **data ordering**. They know that if the data is messy, the logic is hard; if the data is ordered, the logic becomes trivial.

1. Problem Explanation

The Premise: You are given a collection of intervals, where each interval is a pair `[start, end]`. Some of these intervals might overlap (like two meetings scheduled in the same room). Your job is to **merge** all overlapping intervals into a single, continuous interval that covers the same range.

The “Why” of Overlapping: Two intervals overlap if the **start** of one happens **before or exactly when** the **end** of another has occurred.

Example: Input: `[[1, 3], [2, 6], [8, 10], [15, 18]]` 1. `[1, 3]` and `[2, 6]` overlap because 2 is less than 3. They merge into `[1, 6]`. 2. `[1, 6]` and `[8, 10]` do **not** overlap because 8 is greater than 6. 3. Final result: `[[1, 6], [8, 10], [15, 18]]`.

2. Solution Explanation

The secret to solving this efficiently is **Sorting**. If we sort the intervals by their start times, we only ever have to compare the *current* interval with the *last* one we processed.

The Strategy:

1. **Sort** the intervals by their starting value.
2. **Initialize** a list called **merged** with the first interval.
3. **Iterate** through the rest of the intervals one by one:
 - Compare the **start** of the current interval with the **end** of the *last* interval in our **merged** list.
 - **If they overlap:** Update the **end** of the last interval in **merged** to be the maximum of both ends.
 - **If they don’t overlap:** Just add the current interval to the **merged** list as a new entry.

Detailed ASCII Walkthrough Input: `[[1, 4], [4, 5], [2, 6], [8, 10]]`

Step 1: Sort by start time Sorted: `[[1, 4], [2, 6], [4, 5], [8, 10]]`

Step 2: Process [1, 4] merged is empty, so add `[1, 4]`. **merged** = `[[1, 4]]`

Timeline:

```
1 -- 2 -- 3 -- 4 -- 5 -- 6 -- 7 -- 8 -- 9 -- 10
[=====]
```

Step 3: Process [2, 6] Current start (2) <= Last merged end (4). **OVERLAP!** Update last merged end to `max(4, 6) = 6`. **merged** = `[[1, 6]]`

Timeline:

```
1 -- 2 -- 3 -- 4 -- 5 -- 6 -- 7 -- 8 -- 9 -- 10
```

```
[=====]
      (merged 2-6 into 1-4)
```

Step 4: Process [4, 5] Current start (4) $\leq$ Last merged end (6). **OVERLAP!** Update last merged end to $\max(6, 5) = 6$. (It stays 6). `merged = [[1, 6]]`

```
Timeline:
1 -- 2 -- 3 -- 4 -- 5 -- 6 -- 7 -- 8 -- 9 -- 10
[=====]
               [=====] <-- already contained!
```

Step 5: Process [8, 10] Current start (8) $>$ Last merged end (6). **NO OVERLAP.** Append [8, 10] to merged. `merged = [[1, 6], [8, 10]]`

```
Timeline:
1 -- 2 -- 3 -- 4 -- 5 -- 6 -- 7 -- 8 -- 9 -- 10
[=====]           [=====]
```

3. Time and Space Complexity Analysis

Time Complexity: $O(n * \log n)$ Where n is the number of intervals.

TC Derivation:

1. Sorting: Most efficient sorting algorithms (like Timsort) take $O(n * \log n)$.
2. Linear Scan: We walk through the sorted list once. This takes $O(n)$.

Total Time = $O(n * \log n) + O(n)$

Because $n * \log n$ grows faster, we drop the $O(n)$.

Final TC: $O(n * \log n)$

Space Complexity: $O(n)$ (or $O(\log n)$ depending on implementation)

SC Derivation:

1. The 'merged' list: In the worst case (no overlaps), we store all n intervals.
Space: $O(n)$
2. Sorting space: Most internal sort functions use some extra space for the recursion stack or temporary arrays.
Space: $O(\log n)$

Final SC: $O(n)$

4. Solution Code

Python Implementation

```
def merge(intervals):
    if not intervals:
        return []

    # 1. Sort intervals by start time.
    # This is non-trivial because it ensures we only need
    # to look back at the 'last' merged interval.
    intervals.sort(key=lambda x: x[0])

    merged = []
    for interval in intervals:
        # If merged is empty or current start > last merged end,
        # there is no overlap.
        if not merged or interval[0] > merged[-1][1]:
            merged.append(interval)
        else:
            # There is an overlap, merge the current interval
            # with the previous one by updating the end time.
            merged[-1][1] = max(merged[-1][1], interval[1])

    return merged
```

JavaScript Implementation

```
var merge = function(intervals) {
    if (intervals.length <= 1) return intervals;

    // 1. Sort by start time.
    // In JS, sort() converts to string by default,
    // so we must provide a comparator function.
    intervals.sort((a, b) => a[0] - b[0]);

    const merged = [intervals[0]];

    for (let i = 1; i < intervals.length; i++) {
        let current = intervals[i];
        let lastMerged = merged[merged.length - 1];

        // Check for overlap: current start <= last merged end
        if (current[0] <= lastMerged[1]) {
            // Update the end of the last merged interval
            lastMerged[1] = Math.max(lastMerged[1], current[1]);
        } else {
```



```

        // No overlap, add current as a new entry
        merged.push(current);
    }
}

return merged;
};

```

Note 1: Terms and Techniques

- **Sorting with Comparator:** Instead of sorting numbers, we sort “objects” (arrays) based on a specific property (the first element). This is a foundational step in almost all interval-based problems.
 - **In-Place vs. Auxiliary Space:** An L6 might discuss whether to modify the input `intervals` array to save space or return a new one to keep the function “pure” (not changing the input).
-

Note 2: Real-World Interview Variations

1. **Google: “Calendar Free Time” (LeetCode 759) * The Twist:** Given the schedules of several people, find the time blocks where *everyone* is free. * **L5 Solution:** Merge all everyone’s busy intervals into one list. Sort them. The gaps *between* the merged intervals are the free time blocks.
2. **Bloomberg: “Insert Interval” (LeetCode 57) * The Twist:** The list is *already sorted*, and you need to insert one new interval and merge if necessary. * **L5 Solution:** Since it’s already sorted, don’t re-sort! Iterate once, add all intervals that end before the new one starts, merge the new one with any overlaps, and add the rest. This is $O(n)$.
3. **Meta: “Meeting Rooms II” (LeetCode 253) * The Twist:** Find the *minimum* number of conference rooms required for a list of meetings. * **L5 Solution:** This is “Interval Point Counting.” Use a Min-Heap to track the end times of meetings currently in rooms. If a new meeting starts after the earliest end time, you can reuse that room. The max size of the heap is the answer.

435. Non-overlapping Intervals

A senior engineer at Google approaches interval problems by immediately identifying the type of conflict. In this case, we aren’t just merging intervals; we are trying to **maximize** the number of intervals we keep. This is a classic **Greedy Scheduling** problem.

Here is how an L5/L6 engineer would explain and solve LeetCode 435.

1. Problem Explanation

The Goal: You are given an array of intervals where `intervals[i] = [start, end]`. You need to find the **minimum** number of intervals you must remove to make the rest of them non-overlapping.

What is an Overlap? An interval `[1, 2]` and `[2, 3]` are **not** overlapping (they just touch). An interval `[1, 3]` and `[2, 4]` **are** overlapping because 2 is less than 3.

The Logic of “Minimum Removal”: Finding the minimum number of intervals to remove is the exact same thing as finding the **maximum** number of intervals you can keep. * Total Intervals - Max Non-Overlapping = Min Removals.

Visualizing the Problem: Let `intervals = [[1,2], [2,3], [3,4], [1,3]]`

```
Time Line:  1    2    3    4
             |---|          (A: [1,2])
              |---|        (B: [2,3])
               |---|      (C: [3,4])
              |-----|    (D: [1,3])
```

Observation:

Interval D overlaps with both A and B.

If we keep D, we must remove A and B (2 removals).

If we remove D, we keep A, B, and C (1 removal).

Winner: Remove D.

2. Solution Explanation

The Greedy Insight: Why End Times Matter Most people instinctively want to sort by the **start time**. However, a senior engineer knows that to fit as many things as possible into a schedule, you should pick the task that **finishes the earliest**.

Why? Because the earlier a task finishes, the more room you leave for everything else that comes after it.

The Algorithm: 1. **Sort** all intervals by their **end times**. 2. Keep track of the `prev_end` (the end time of the last interval we decided to keep). 3. Iterate through the sorted intervals: * If the current interval starts **after or at** `prev_end`, it doesn't overlap! We keep it and update `prev_end` to the current interval's end. * If the current interval starts **before** `prev_end`, it overlaps. We must remove it. Increment our removal counter.

ASCII Diagram Walkthrough Let `intervals = [[1,10], [2,3], [3,4], [5,6]]`

Step 1: Sort by End Time

Sorted: `[[2,3], [3,4], [5,6], [1,10]]`

Step 2: Processing

1. Current: `[2,3]`
 - First interval, always keep it.
 - `prev_end = 3`
 - `Removals = 0`
2. Current: `[3,4]`
 - Does it start `>= prev_end (3)`? Yes (`3 >= 3`).
 - Keep it. Update `prev_end = 4`.
 - `Removals = 0`
3. Current: `[5,6]`
 - Does it start `>= prev_end (4)`? Yes (`5 >= 4`).
 - Keep it. Update `prev_end = 6`.
 - `Removals = 0`
4. Current: `[1,10]`
 - Does it start `>= prev_end (6)`? No (`1 < 6`).
 - **OVERLAP DETECTED!**
 - Action: Remove `[1,10]`.
 - `Removals = 1`

Final Answer: 1 removal.

3. Time and Space Complexity Analysis

+-----+-----+	
	TIME COMPLEXITY DERIVATION: $O(N \log N)$
+-----+-----+	
	N = Number of intervals
	1. Sorting Phase:
	We sort the intervals by end time.
	Cost: $O(N \log N)$
	2. Linear Scan Phase:
	We iterate through the sorted list exactly once.
	Cost: $O(N)$

TOTAL TIME: $O(N \log N) + O(N)$ =====> $O(N \log N)$
SPACE COMPLEXITY DERIVATION: $O(1)$ or $O(N)$
1. In-place Sorting: Depending on the language (like Python's Timsort), sorting can take $O(N)$ extra space.
2. Working Variables: We only store 'prev_end' and 'removals' (integers). Cost: $O(1)$
TOTAL SPACE: $O(1)$ auxiliary space (excluding sort overhead).

4. Solution Code

Python Snippet

```
def eraseOverlapIntervals(intervals):
    if not intervals:
        return 0

    # Sort by the end time (index 1)
    # This is the "Greedy" choice: finish as early as possible
    intervals.sort(key=lambda x: x[1])

    removals = 0
    # Initialize prev_end with the end of the first interval
    prev_end = intervals[0][1]

    # Start from the second interval
    for i in range(1, len(intervals)):
        start, end = intervals[i]

        # If the current interval starts before the previous one ends
        if start < prev_end:
            # We must remove this one to keep the one that ends earlier
            removals += 1
        else:
            # No overlap, update the prev_end to current
            prev_end = end
```

```
return removals
```

JavaScript Snippet

```
function eraseOverlapIntervals(intervals) {
    if (intervals.length === 0) return 0;

    // Sort by end time (the second element in the sub-array)
    intervals.sort((a, b) => a[1] - b[1]);

    let removals = 0;
    // Track the end time of the last non-overlapping interval we kept
    let prevEnd = intervals[0][1];

    for (let i = 1; i < intervals.length; i++) {
        let currentStart = intervals[i][0];
        let currentEnd = intervals[i][1];

        if (currentStart < prevEnd) {
            // Overlap! Since we sorted by end time, we know the
            // current interval ends AFTER or equal to prevEnd.
            // To keep the interval that ends earlier, we "remove" current.
            removals++;
        } else {
            // No overlap, update our boundary
            prevEnd = currentEnd;
        }
    }

    return removals;
}
```

Note 1: Terminology and Techniques

- **Greedy Algorithm:** An algorithmic paradigm that follows the problem-solving heuristic of making the locally optimal choice at each stage with the intent of finding a global optimum. In this problem, the “local choice” is picking the interval that finishes first.
 - **Interval Scheduling:** A classic class of problems involving a set of tasks with start and end times. The goal is usually to find the maximum number of tasks that can be performed by a single person/machine.
-

Note 2: Real-World Interview Variations

1. Google: “Meeting Room Scheduler” * Prompt: You have one conference room and a list of meeting requests. What is the maximum number of meetings you can hold? * **Solution:** This is exactly LC 435. Instead of returning `removals`, you return `Total - removals` (the count of intervals you kept).

2. Meta: “Ad Campaign Optimization” * Prompt: You are given several time slots where different ads can run. Some overlap. You want to run the maximum number of ads possible, but you can only run one at a time. * **Solution:** Sort by end time. Iterate and pick every ad that starts after the previous one finishes.

3. Bloomberg: “Trade Execution Windows” * Prompt: A trader has several windows of time where they can execute a specific trade. Some windows overlap. They want to execute as many trades as possible, but each execution requires their full attention for the entire window. * **Solution:** Again, the “earliest deadline first” (Earliest End Time) greedy strategy ensures the trader is free as soon as possible to take the next trade window.

Meeting rooms

When a senior engineer (L5 or L6) at Google or Meta tackles the “Meeting Rooms” problem (LeetCode 252), they aren’t just looking for a “True” or “False.” They are looking for **Interval Geometry**.

The key insight for a senior engineer is realizing that the problem of “Can a person attend all meetings?” is mathematically identical to “Is there any overlap between these intervals?” To solve this efficiently, we must impose order on chaos.

1. Problem Explanation

The Core Question: Given an array of meeting time intervals `[[start_i, end_i], ...]`, determine if a person could attend all meetings.

The Golden Rule: You can only be in one place at a time. If Meeting A ends at 10:00 AM, you can start Meeting B at 10:00 AM. However, if Meeting A ends at 10:00 AM and Meeting B starts at 9:50 AM, you have a conflict.

Visualizing the “Why”:

Case A: No Overlap (Valid) Meetings: `[[0, 5], [10, 15], [20, 25]]`

Time: 0----5-----10----15-----20----25
M1: [====]
M2: [=====]

M3: [====]

Result: TRUE. There is "breathing room" between every block.

Case B: Overlap (Invalid) Meetings: [[0, 30], [5, 10], [15, 20]]

Time: 0----5----10----15----20----25----30

M1: [=====]

M2: [====] <-- CONFLICT!

M3: [====] <-- CONFLICT!

Result: FALSE. Meeting 1 swallows the time slots for M2 and M3.

2. Solution Explanation

If we look at the meetings in a random order, we have to compare every meeting against every other meeting. That would be very slow.

The Senior Strategy: The Sorted Timeline By sorting the meetings based on their **start times**, we transform a 2D “collision” problem into a 1D “sequential” check. Once sorted, we only ever need to compare a meeting with the one immediately following it.

The Rule of Overlap: For any two adjacent meetings in a sorted list: Meeting A: [startA, endA] Meeting B: [startB, endB]

An overlap exists **IF AND ONLY IF**: startB < endA

Step-by-Step Walkthrough: Input: [[15, 20], [0, 30], [5, 10]]

Step 1: Sort by start time.

Before: [[15, 20], [0, 30], [5, 10]]

After: [[0, 30], [5, 10], [15, 20]]

Step 2: Compare pairs.

Pair 1: [0, 30] and [5, 10]

Is 5 < 30?

YES.

Conclusion: Overlap detected. Return FALSE immediately.

3. Time and Space Complexity Analysis

Let N be the number of meetings in the input list.

=====

TIME COMPLEXITY: $O(N \log N)$

=====

Derivation Diagram:

```
[ Unsorted Meetings ]
      |
      v
[ Sorting Step ] -----> O(N log N)
  | (Using Timsort or Quicksort)
  |
  v
[ Sequential Scan ] -----> O(N)
  | (Single loop through N-1 pairs)
  | Compare current.end vs next.start
  |
  v
Total Time = O(N log N) + O(N)
The N log N term is the bottleneck (dominant).
Total Time = O(N log N).
```

=====

SPACE COMPLEXITY: O(1) or O(N)

=====

Derivation Diagram:

```
[ In-place Sort ] -----> O(1) extra space
  | (If we are allowed to modify the input)

[ Sort with Copy ] -----> O(N) extra space
  | (If we must preserve the original input)

[ Comparison Loop ] -----> O(1) extra space
  | (Only using one index/pointer)
```

Conclusion: Most interviewers accept O(1) if you sort in-place,
but technically sorting itself takes O(log N) stack space in some
implementations. We usually call this O(1) auxiliary space.

=====

4. Solution Code

JavaScript

```
/**
 * L5/L6 Approach: Sorting + Greedy Linear Scan
 * Time: O(N log N) | Space: O(1)
 */
```



```

function canAttendMeetings(intervals) {
    // 1. Sort the intervals by start time.
    // If start times are equal, the order doesn't strictly
    // matter for this specific overlap check.
    intervals.sort((a, b) => a[0] - b[0]);

    // 2. Iterate through the sorted intervals, starting from the second one.
    for (let i = 1; i < intervals.length; i++) {
        let currentMeeting = intervals[i];
        let previousMeeting = intervals[i - 1];

        // Check for overlap: If the current meeting starts
        // before the previous one finished.
        if (currentMeeting[0] < previousMeeting[1]) {
            return false;
        }
    }

    // 3. If we finish the loop without finding an overlap, it's valid.
    return true;
}

```

Python

```

# L5/L6 Approach: Sorting + Greedy Linear Scan
# Time:  $O(N \log N)$  | Space:  $O(1)$ 
def canAttendMeetings(intervals):
    # 1. Sort the intervals by start time.
    # Python's sort() is highly optimized Timsort.
    intervals.sort(key=lambda x: x[0])

    # 2. Sequential check of neighboring meetings.
    for i in range(1, len(intervals)):
        # If the start time of the current meeting (intervals[i][0])
        # is earlier than the end time of the last meeting (intervals[i-1][1])
        if intervals[i][0] < intervals[i-1][1]:
            return False

    # 3. No overlaps found.
    return True

```

Note 1: Terms and Techniques

- **Greedy Algorithm:** We make the locally optimal choice (sorting the timeline) to simplify the global problem.

- **Interval Merging / Overlap Logic:** This is a fundamental pattern in computer science used in calendar apps, memory management, and rendering (z-buffering).
- **In-place vs. Out-of-place:** A senior engineer will always ask the interviewer: “Do you mind if I modify the input array?” Sorting the input `intervals` array saves space $O(1)$ but “destroys” the user’s original data order.

Note 2: Real-World Interview Variations

1. **“Meeting Rooms II” (Google / Bloomberg)** * **The Problem:** Instead of True/False, find the *minimum number of rooms* required to hold all meetings. * **The Solution:** Use a **Min-Heap**. Sort meetings by start time. For each meeting, check the top of the heap (the room that finishes earliest). If the earliest room is free, “reuse” it by updating its end time in the heap. If not, “add a room” by pushing the current meeting’s end time into the heap. The final heap size is your answer.
2. **“Merge Intervals” (Meta)** * **The Problem:** Given $[[1,3], [2,6], [8,10]]$, merge overlapping ones into $[[1,6], [8,10]]$. * **The Solution:** Sort by start time. Iterate and maintain a `merged` list. If the current interval overlaps with the last one in `merged`, update the end time of the last one to be the `max(last.end, current.end)`.
3. **“Employee Free Time” (Google)** * **The Problem:** Given schedules for multiple employees, find the gaps where *everyone* is free. * **The Solution:** Flatten all intervals into one big list, sort them, and perform a merge. The “gaps” between the merged intervals are the free times. A senior engineer would optimize this by using a **Heap** to merge k sorted lists (one for each employee) in $O(N \log K)$ time.

Meeting rooms II

A Google L5/L6 engineer views “Meeting Rooms II” as a **Resource Allocation** problem. In a production environment, this is exactly how a load balancer decides how many server instances to spin up based on incoming request “start” and “end” times.

The key to solving this at a senior level is recognizing that we don’t care which meeting is in which room; we only care about the **peak number of concurrent overlaps**.

1. Problem Explanation

The Goal: Given an array of meeting time intervals `[[start_1, end_1], [start_2, end_2], ...]`, find the minimum number of conference rooms required to hold all meetings.

The Logic: If two meetings overlap, they need separate rooms. If a meeting ends before or at the exact time another starts, the second meeting can “reuse” the first room.

The Intuition: Imagine a hotel lobby with a digital clock. * Every time a meeting **starts**, someone walks in and asks for a room key. * Every time a meeting **ends**, someone walks to the desk and returns a key. The “Minimum Rooms” is simply the **maximum number of keys out at any given time**.

2. Solution Explanation

There are two high-level ways to solve this. An L5+ engineer will usually prefer the **Chronological Ordering** (Two-Pointer) approach because it is extremely stable and avoids the overhead of complex data structures like Heaps if not strictly necessary.

The Chronological Ordering Strategy Instead of looking at “Intervals,” we break every interval into two separate **events**: a Start Event and an End Event.

1. Collect all Start times and sort them: `[0, 5, 15]`
2. Collect all End times and sort them: `[10, 20, 30]`
3. Use two pointers to walk through time.

Step-by-Step Visualization Input: `[[0, 30], [5, 10], [15, 20]]`

```
=====
INITIAL STATE
=====
Starts (Sorted): [0, 5, 15]
Ends (Sorted):   [10, 20, 30]

Pointers:
  s_ptr = 0 (pointing at Start 0)
  e_ptr = 0 (pointing at End 10)

Rooms_In_Use: 0
Peak_Rooms:    0

=====
STEP 1: Time = 0 (A meeting starts)
```

```

=====
Current Start: 0
Current End: 10

Since Start (0) < End (10):
- Someone needs a room NOW.
- Rooms_In_Use: 1
- Peak_Rooms: 1 (Max of 0 and 1)
- Move s_ptr to next Start (5)

=====
STEP 2: Time = 5 (A meeting starts)
=====
Current Start: 5
Current End: 10

Since Start (5) < End (10):
- Another meeting starts before the first one ends!
- Rooms_In_Use: 2
- Peak_Rooms: 2 (Max of 1 and 2)
- Move s_ptr to next Start (15)

=====
STEP 3: Time = 10 (A meeting ends)
=====
Current Start: 15
Current End: 10

Since End (10) <= Start (15):
- A meeting finished! Someone returned a key.
- Rooms_In_Use: 1
- Move e_ptr to next End (20)

=====
STEP 4: Time = 15 (A meeting starts)
=====
Current Start: 15
Current End: 20

Since Start (15) < End (20):
- A new meeting starts. (It likely reused the room from Step 3).
- Rooms_In_Use: 2
- Peak_Rooms: 2 (Max of 2 and 2)
- Move s_ptr (End of Start list)
=====

```

CONCLUSION

=====

We process the remaining Ends. Rooms_In_Use will decrease, but Peak_Rooms will stay at 2.

Final Result: 2

3. Time and Space Complexity Analysis

Let N be the number of meetings.

TIME COMPLEXITY DERIVATION: $O(N \log N)$

-
- | | |
|---|---------------------------------------|
| 1. Extracting starts and ends into lists: | $O(N)$ |
| 2. Sorting the Starts list: | $O(N \log N)$ |
| 3. Sorting the Ends list: | $O(N \log N)$ |
| 4. Two-pointer traversal: | $O(N)$ (each pointer moves N times) |

Total Time = $O(N) + O(N \log N) + O(N \log N) + O(N)$

Dominant term: $O(N \log N)$

SPACE COMPLEXITY DERIVATION: $O(N)$

-
- | | |
|------------------|--------------|
| 1. Starts array: | $O(N)$ space |
| 2. Ends array: | $O(N)$ space |

Total Space = $O(N) + O(N) = O(N)$

(Note: In-place sorting might reduce this slightly, but usually we count the auxiliary arrays).

4. Solution Code

Optimized Approach: Chronological Ordering (Two-Pointer) This is preferred for its simplicity and clear logic.

Python

```
def minMeetingRooms(intervals):
    if not intervals:
        return 0

    # Separate and sort start and end times
    start_times = sorted([i[0] for i in intervals])
    end_times = sorted([i[1] for i in intervals])
```

```

used_rooms = 0
max_rooms = 0
s_ptr = 0
e_ptr = 0

# Iterate through all meetings using the start pointer
while s_ptr < len(intervals):
    # If a meeting starts before the oldest meeting ends,
    # we need a new room.
    if start_times[s_ptr] < end_times[e_ptr]:
        used_rooms += 1
        s_ptr += 1
    else:
        # A meeting ended, we can reuse a room.
        # We don't decrement used_rooms immediately;
        # we just move the end pointer to 'free up' a slot.
        used_rooms -= 1
        e_ptr += 1

    # Track the highest number of concurrent rooms
    max_rooms = max(max_rooms, used_rooms)

return max_rooms

```

JavaScript

```

function minMeetingRooms(intervals) {
    if (!intervals || intervals.length === 0) return 0;

    const starts = intervals.map(i => i[0]).sort((a, b) => a - b);
    const ends = intervals.map(i => i[1]).sort((a, b) => a - b);

    let rooms = 0;
    let maxRooms = 0;
    let sPtr = 0;
    let ePtr = 0;

    while (sPtr < intervals.length) {
        if (starts[sPtr] < ends[ePtr]) {
            // New meeting starts
            rooms++;
            sPtr++;
        } else {
            // Meeting ended
            rooms--;
        }
    }
    return maxRooms;
}

```

```

        ePtr++;
    }
    maxRooms = Math.max(maxRooms, rooms);
}

return maxRooms;
}

```

Note 1: Terminology & Techniques

- **Chronological Ordering:** Instead of treating an interval as a single object, we treat its boundaries as discrete events on a timeline. This is a common pattern for “Sweep Line” algorithms.
 - **Two-Pointer Technique:** Using two variables to track progress through two different sorted lists simultaneously. It allows us to simulate the passage of time without a granular “second-by-second” loop.
-

Note 2: Real-World Interview Variations

1. Variation: Maximum Number of Guests (Bloomberg) * The Problem: Given entry and exit times of guests at a party, find the time when the most guests were present. *** The Solution:** Exactly the same algorithm. Instead of returning `max_rooms`, you return the `start_time` that triggered the `max_rooms` count.

2. Variation: CPU Task Scheduling (Google) * The Problem: You have tasks with start times and durations. You have K cores. What is the minimum number of cores to finish all tasks without delay? *** The Solution:** This is Meeting Rooms II in disguise. `Start = start_time`, `End = start_time + duration`.

3. Variation: Airplane Parking (Meta) * The Problem: Planes land and take off at certain times. How many gates are needed? *** The Solution:** Same logic. A senior engineer would also ask about “Buffer time” (e.g., a gate needs 15 mins to clean between planes). *** Refined Logic:** You would adjust the end times: `end_times = sorted([i[1] + 15 for i in intervals])`. This shows you think about real-world constraints.

4. Variation: Carpooling (Uber/Lyft) * The Problem: A car has capacity seats. Given trips `[num_people, start, end]`, can you pick up everyone? *** The Solution:** Use the same “Sweep Line” approach. Instead of `rooms += 1`, you do `current_passengers += num_people`. If `current_passengers > capacity` at any point, return False.

1851. Minimum Interval to Include Each Query

Solving “1851. Minimum Interval to Include Each Query” at an L5/L6 level requires recognizing that this isn’t just a search problem—it is a **Sweep Line** and **Priority Queue** problem. A senior engineer identifies that processing queries in their original order is inefficient; instead, we should process them in a way that matches the “flow” of the intervals.

1. Problem Explanation

The Goal: You are given a set of intervals `[left, right]` and a set of `queries`. For each query point, you need to find the **smallest interval** (measured by `right - left + 1`) that contains that point. If no such interval exists, return -1.

The Complexity: Imagine you have many overlapping intervals of different lengths. For every single query, searching through all intervals is too slow.

Example: Intervals: `[[1, 4], [2, 4], [3, 6]]` Lengths: `[4, 3, 4]` (Size = `right - left + 1`) Query: 4 The intervals containing 4 are `[1, 4]`, `[2, 4]`, and `[3, 6]`. The minimum length is 3 (from `[2, 4]`).

2. Solution Explanation

The L5/L6 strategy uses a **Sorted Query Sweep** with a **Min-Heap**.

Phase 1: Preparation

1. **Sort the Intervals:** Sort them by their start time. This allows us to “activate” them only when a query reaches their start.
2. **Sort the Queries:** This is the most important step. If we sort queries, we only move “forward” through the intervals. We keep track of the original indices so we can return the answer in the correct order.

Phase 2: The Sweep Line We iterate through the sorted queries. For each query: 1. **Add Intervals:** Add all intervals that *start* before or at the query point into a Min-Heap. 2. **Heap Contents:** The Min-Heap stores pairs: `[interval_length, interval_right_boundary]`. 3. **Clean Up:** Remove intervals from the top of the heap if their *right boundary* is less than the current query (meaning they are already “behind” us). 4. **Result:** The top of the heap is the smallest valid interval for this query!

Step-by-Step ASCII Visualization:

Intervals: `[[1, 4], [2, 4], [3, 6]]`
Queries: `[2, 3, 5]`


```

=====
STEP 1: Sort and Prepare
=====
Intervals (sorted by start): [[1, 4], [2, 4], [3, 6]]
Queries (sorted with indices): [(2, idx:0), (3, idx:1), (5, idx:2)]
Min-Heap: []

=====
STEP 2: Process Query = 2
=====
1. Add intervals starting <= 2:
   - [1, 4] (Length 4). Heap: [[4, end:4]]
   - [2, 4] (Length 3). Heap: [[3, end:4], [4, end:4]]

2. Clean up (end < 2):
   - None to remove.

3. Result for Query 2: Top of Heap is length 3.
   Ans[idx:0] = 3

=====
STEP 3: Process Query = 3
=====
1. Add intervals starting <= 3:
   - [3, 6] (Length 4). Heap: [[3, end:4], [4, end:4], [4, end:6]]

2. Clean up (end < 3):
   - None to remove.

3. Result for Query 3: Top of Heap is length 3.
   Ans[idx:1] = 3

=====
STEP 4: Process Query = 5
=====
1. Add intervals starting <= 5:
   - No new intervals start.

2. Clean up (end < 5):
   - Heap top is [3, end:4]. Since 4 < 5, REMOVE.
   - Heap top is [4, end:4]. Since 4 < 5, REMOVE.
   - Heap top is [4, end:6]. Since 6 >= 5, KEEP.

3. Result for Query 5: Top of Heap is length 4.
   Ans[idx:2] = 4

```

```
=====
FINAL RESULT
=====
```

```
Ans: [3, 3, 4]
```

3. Time and Space Complexity Analysis

Let N be the number of intervals and Q be the number of queries.

TIME COMPLEXITY: $O(N \log N + Q \log Q)$

1. Sorting Intervals: $O(N \log N)$
2. Sorting Queries: $O(Q \log Q)$
3. Main Loop (Sweep):
 - Each interval is pushed to heap once: $O(N \log N)$
 - Each interval is popped from heap once: $O(N \log N)$
 - For each query, we peek the heap: $O(Q \log N)$

Total Time = $(N \log N) + (Q \log Q) + (N \log N) + (Q \log N)$
Since N and Q can be similar, it is $O((N+Q) \log N)$.

Total Time = $O((N+Q) \log N)$

SPACE COMPLEXITY: $O(N + Q)$

Memory allocated:

1. Sorted Queries array: $O(Q)$
2. Min-Heap: Worst case all intervals inside: $O(N)$
3. Answer array: $O(Q)$

Total Space = $O(N + Q)$

4. Solution Code

Python Solution (Using heapq)

```
import heapq

def minInterval(intervals, queries):
    # Sort intervals by start time
    intervals.sort()
```

```

# Sort queries but keep track of original indices to return answer in order
sorted_queries = sorted([(q, i) for i, q in enumerate(queries)])

res = [-1] * len(queries)
min_heap = [] # Stores [length, end_point]
i = 0 # Pointer for intervals

for q_val, original_idx in sorted_queries:
    # Step 1: Add all intervals that start before or at current query
    while i < len(intervals) and intervals[i][0] <= q_val:
        l, r = intervals[i]
        # [Length, End_boundary]
        heapq.heappush(min_heap, [r - l + 1, r])
        i += 1

    # Step 2: Remove intervals that end before the current query
    while min_heap and min_heap[0][1] < q_val:
        heapq.heappop(min_heap)

    # Step 3: Top of heap is the smallest valid interval
    if min_heap:
        res[original_idx] = min_heap[0][0]

return res

```

JavaScript Solution (With Min-Heap Logic)

```

/**
 * @param {number[][]} intervals
 * @param {number[]} queries
 * @return {number[]}
 */
var minInterval = function(intervals, queries) {
    // Sort intervals by start point
    intervals.sort((a, b) => a[0] - b[0]);

    // Create sorted queries with indices
    const sortedQueries = queries
        .map((val, idx) => [val, idx])
        .sort((a, b) => a[0] - b[0]);

    const res = new Array(queries.length).fill(-1);
    const minHeap = new MinPriorityQueue({ priority: (x) => x[0] });

    let i = 0;
    for (const [qVal, originalIdx] of sortedQueries) {

```

```

// Step 1: Push valid starting intervals
while (i < intervals.length && intervals[i][0] <= qVal) {
    const [l, r] = intervals[i];
    minHeap.enqueue([r - l + 1, r]);
    i++;
}

// Step 2: Pop intervals that are already past the query point
while (!minHeap.isEmpty() && minHeap.front().element[1] < qVal) {
    minHeap.dequeue();
}

// Step 3: Front is the smallest
if (!minHeap.isEmpty()) {
    res[originalIdx] = minHeap.front().element[0];
}
}

return res;
};

```

Note 1: Terminology and Techniques

- **Sweep Line Algorithm:** A technique where you process “events” across a 1D or 2D space in a sorted order. It converts a static spatial problem into a dynamic temporal problem.
 - **Offline Queries:** When you know all queries in advance, you can sort them to optimize processing. This is “offline” because you aren’t forced to answer them as they arrive.
 - **Priority Queue (Min-Heap):** A data structure that allows you to always access the smallest element in logarithmic time. Here, it maintains the smallest interval length currently covering our “sweep line”.
-

Note 2: Real-World / Interview Variations

1. Meeting Rooms II (Google / Meta) * The Problem: Given meeting times, find the minimum number of rooms needed. *** L5 Solution:** Use a Sweep Line. Sort all start and end times. When a meeting starts, `rooms++`. When it ends, `rooms--`. The answer is the maximum value `rooms` ever reaches. This tests the same “sorted events” logic.

2. Ad Attribution Windows (Meta / Ads Engineering) * The Context: An ad is shown at time X. A purchase happens at time Y. Find the smallest “attribution window” (interval) that links the two. *** L5 Solution:** Similar to

the sweep line here. You sort events by timestamp and use a sliding window or priority queue to find the tightest interval that contains both the “view” and the “conversion” event.

3. Server Capacity Planning (Bloomberg) * The Context: Each task has a duration [`start`, `end`] and a resource requirement. At any point in time, what is the total resource load? * **L5 Solution:** This is the “Sweep Line” again. Sort the starts and ends. At each event point, update the total load. If the question asks for the smallest server that can handle a specific task among many, you use the Min-Heap approach described in this Leetcode solution.

228. Summary Ranges

At the Google L5/L6 level, engineers look for the most “robust” way to handle edge cases—like a list with one element or a sequence that ends abruptly. They avoid complex nested logic that is hard to debug and instead favor a clean, linear “Two-Pointer” or “Scanning” approach.

1. Problem Explanation

The Goal: You are given a **sorted** list of unique integers. Your job is to find groups of numbers that are consecutive (they increase by exactly 1). * If numbers are consecutive, you format them as "`start->end`". * If a number is all by itself (the numbers around it aren't consecutive), just list the number as a string.

The “Why” of the Logic: Since the input is already sorted, we don't need to jump around. We just need to identify where a “link” in the chain breaks. A break happens whenever `current_number + 1` is **not** equal to the `next_number`.

2. Solution Explanation

We use a “Fast Pointer” approach. We keep a fixed point at the **start** of a potential range and move a second pointer `i` forward as long as the numbers are consecutive.

The Intuition:

1. Start at the first number. This is your **start** of the range.
2. Look at the next number. Is it `current + 1`?
 - **Yes:** Keep going.
 - **No:** The range is over! Format the range from **start** to **current**, add it to the result, and reset **start** to the next number.

Step-by-Step Visualization Input: [0, 1, 2, 4, 5, 7]

```
=====
INITIAL STATE
=====
Array: [0, 1, 2, 4, 5, 7]
Index: 0 1 2 3 4 5
Result: []

=====
STEP 1: Starting at Index 0
=====
Start Value: 0
Current pointer (i): 0

Is nums[i+1] == nums[i] + 1?
- 1 == 0 + 1? YES. Move i to 1.
- 2 == 1 + 1? YES. Move i to 2.
- 4 == 2 + 1? NO! (Break found at index 2)

Action: We have a range from start(0) to nums[2](2).
Format: "0->2"
Result: ["0->2"]

=====
STEP 2: Start at Index 3
=====
New Start Value: 4
Current pointer (i): 3

Is nums[i+1] == nums[i] + 1?
- 5 == 4 + 1? YES. Move i to 4.
- 7 == 5 + 1? NO! (Break found at index 4)

Action: We have a range from start(4) to nums[4](5).
Format: "4->5"
Result: ["0->2", "4->5"]

=====
STEP 3: Start at Index 5
=====
New Start Value: 7
Current pointer (i): 5

Is there a next number? NO. (End of array reached)
```

Action: start(7) is equal to nums[5](7). This is a single number.
Format: "7"
Result: ["0->2", "4->5", "7"]

```
=====
FINAL OUTPUT: ["0->2", "4->5", "7"]
=====
```

3. Time and Space Complexity Analysis

TIME COMPLEXITY DERIVATION: $O(n)$

Input size: n elements.

We use a 'while' loop that starts at index 0.
Inside, we might have another 'while' loop to find the end
of the range, BUT the pointer 'i' only ever moves forward.

Each element in the array is visited exactly ONCE by the
main pointer.

Calculation:

Total Operations = n (visits) * constant time work per visit
Total Time = Order of n

SPACE COMPLEXITY DERIVATION: $O(n)$

What are we storing?

1. The result list: In the worst case (no numbers are consecutive),
we store every number as a separate string.
Space = n strings = Order of n.

2. Auxiliary variables:
We only use 'i' and 'start' (integers).
Space = constant = Order of 1.

Total Space = Order of n (to store the output)

4. Solution Code

Python Implementation

```
def summaryRanges(nums):  
    result = []
```

```

i = 0

while i < len(nums):
    # The beginning of a potential range
    start = nums[i]

    # Move i forward as long as the next element is consecutive
    # We must check i + 1 < len(nums) to prevent index errors
    while i + 1 < len(nums) and nums[i + 1] == nums[i] + 1:
        i += 1

    # If start is equal to the current nums[i], it's a single number
    if start == nums[i]:
        result.append(str(start))
    else:
        # Otherwise, format as a range
        result.append(str(start) + "->" + str(nums[i]))

    # Move to the next element for the next iteration
    i += 1

return result

```

JavaScript Implementation

```

/**
 * @param {number[]} nums
 * @return {string[]}
 */
var summaryRanges = function(nums) {
    const result = [];
    let i = 0;

    while (i < nums.length) {
        // Mark the start of the current range
        const start = nums[i];

        // Scan ahead to find the end of the consecutive sequence
        while (i + 1 < nums.length && nums[i + 1] === nums[i] + 1) {
            i++;
        }

        // Check if the range has more than one number
        if (start === nums[i]) {
            result.push(start.toString());
        } else {

```



```

        result.push(start.toString() + "->" + nums[i].toString());
    }

    // Advance to the next starting point
    i++;
}

return result;
};

```

Note 1: Terminology and Techniques

- **Two-Pointer (Fast/Slow):** While we used one index `i` and a variable `start`, this is a variation of the two-pointer technique. One pointer marks the boundary, and the other explores the extent of the condition.
 - **Linear Scan:** This is the process of moving through a data structure in a single direction. It's efficient because it takes advantage of the "Sorted" property.
-

Note 2: Real World / Interview Variations

1. Variation: Missing Ranges (Google) * The Twist: Instead of grouping what is *there*, group what is *missing* between 0 and 99. Input: [0, 1, 3, 50, 75]. * **How to solve:** Iterate from lower to upper. If `nums[i] > current_expected`, there is a gap. The range is `current_expected -> nums[i] - 1`.

2. Variation: Merge Intervals (Meta / Bloomberg) * The Twist: You have overlapping ranges like [[1, 3], [2, 6], [8, 10]]. Merge them into [[1, 6], [8, 10]]. * **How to solve:** Sort by start time. Keep a "current merged interval." If the next interval starts before the current one ends, update the current one's end time to the maximum of the two.

3. Variation: Data Compression (System Design Context) * The Twist: You are sending a massive list of IDs over a network. How do you reduce the payload size? * **How to solve:** Use the Summary Ranges logic! Instead of sending [101, 102, 103, 104], you send 101:4 (start and length). This is essentially how **Run-Length Encoding (RLE)** works in image and data compression.

452. Minimum Number of Arrows to Burst Balloons

This is an end-to-end, L5/L6 level walkthrough for “Minimum Number of Arrows to Burst Balloons.”

A Senior or Staff Engineer views this not as a geometry problem, but as an **Interval Scheduling Optimization** problem. At this level, the focus is on identifying the exact “Greedy” property that guarantees the global minimum with a single pass. The difference between an L4 and an L6 here is the L6’s immediate recognition that sorting by **end coordinates** is more robust than sorting by start coordinates.

1. Problem Explanation

The Goal: You have balloons represented as 2D intervals on an x-axis (e.g., `[start, end]`). You shoot arrows vertically upwards from any point on the x-axis. An arrow shot at `x` bursts all balloons where `start <= x <= end`. You want to find the **minimum** number of arrows to burst every balloon.

The Intuition: Think of this as finding the maximum number of **non-overlapping** intervals. If two balloons overlap, you can “double-dip” and hit both with one arrow. To minimize arrows, you must maximize how many balloons each arrow hits.

The “Why” of the Challenge: If you have three balloons: A, B, and C. A overlaps with B, and B overlaps with C. Does that mean one arrow hits all three? **Not necessarily.** A and C might not overlap each other!

2. Solution Explanation

To solve this optimally, we use a **Greedy Algorithm**.

The Strategy: Sort by End Point

1. **Sort** all balloons based on their **end** coordinate.
2. **Shoot** the first arrow at the very end of the first balloon. Why? Because shooting it as far right as possible gives it the best chance to hit future balloons that start later.
3. **Check** subsequent balloons. If a balloon starts *before or at* the current arrow’s position, it’s already burst.
4. **Update:** If a balloon starts *after* the current arrow’s position, we need a new arrow. We place this new arrow at the **end** of this new balloon.

Let’s visualize this with: `[[10,16], [2,8], [1,6], [7,12]]`

```

=====
STEP 1: SORTING BY END POINT
=====

Original: [[10,16], [2,8], [1,6], [7,12]]

Sorted by End:
1. [1, 6]   (Ends at 6)
2. [2, 8]   (Ends at 8)
3. [7, 12]  (Ends at 12)
4. [10, 16] (Ends at 16)

=====

STEP 2: VISUALIZING THE OVERLAP
=====

X-axis: 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16
-----
B1:      [=====] (1-6)
B2:      [=====] (2-8)
B3:              [=====] (7-12)
B4:                      [=====] (10-16)

=====

STEP 3: THE SHOOTING PROCESS
=====

--- Arrow 1 ---
We look at B1 [1, 6].
Shoot Arrow 1 at x = 6 (the end of B1).

Does it hit B2 [2, 8]?
Yes, because B2 starts at 2, and 2 <= 6.
B1 and B2 are BURST.

X-axis: 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16
          |
          ARROW 1 (at 6)

--- Arrow 2 ---
Next available balloon is B3 [7, 12].
Does Arrow 1 hit B3?
No, because B3 starts at 7, and 7 > 6.
We need a NEW arrow.
Shoot Arrow 2 at x = 12 (the end of B3).

Does it hit B4 [10, 16]?
Yes, because B4 starts at 10, and 10 <= 12.

```

X-axis: 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16
 |
 ARROW 2 (at 12)

3. Time and Space Complexity Analysis

TIME COMPLEXITY DERIVATION: $O(N * \log N)$

- Total Time = $O(N * \log N) + O(N)$
Dominant term: $O(N * \log N)$

SPACE COMPLEXITY DERIVATION: $O(1)$ or $O(N)$

- Generally, we say $O(1)$ auxiliary space if we sort in-place, otherwise $O(N)$ to store the sorted copy.

4. Solution Code

Python Solution

```
def findMinArrowShots(points):
    if not points:
        return 0

    # SORTING: Crucial L5/L6 detail.
    # Sorting by end (x[1]) is more intuitive for greedy interval problems.
    points.sort(key=lambda x: x[1])

    arrows = 1
    # Place the first arrow at the end of the first balloon
    current_arrow_pos = points[0][1]

    for i in range(1, len(points)):
        start, end = points[i]

        # If the next balloon starts AFTER our current arrow position,
        # the current arrow cannot burst it.
        if start > current_arrow_pos:
            arrows += 1
            # Move the arrow position to the end of this new balloon
            current_arrow_pos = end

    return arrows
```

JavaScript Solution

```
/**
 * @param {number[][]} points
 * @return {number}
 */
var findMinArrowShots = function(points) {
    if (points.length === 0) return 0;

    // Sort by the end coordinate (index 1)
    points.sort((a, b) => a[1] - b[1]);

    let arrows = 1;
    let currentArrowPos = points[0][1];

    for (let i = 1; i < points.length; i++) {
        let start = points[i][0];
        let end = points[i][1];
```

```

        // Non-trivial logic:
        // Because we sorted by end-points, if the 'start' of the
        // current balloon is > 'currentArrowPos', there is NO WAY
        // any previous arrow could have hit this balloon.
        if (start > currentArrowPos) {
            arrows++;
            currentArrowPos = end;
        }
    }

    return arrows;
};

```

Note 1: Terminology & Techniques

- **Greedy Choice Property:** This is the logic that says “if we make the best local decision right now (shooting at the end of the current balloon), it will lead to the best global solution.”
 - **Interval Scheduling:** A class of problems involving overlapping time/space blocks. Sorting is almost always the first step.
-

Note 2: Real World / Interview Variations

1. Variation: Meeting Rooms II (Google/Bloomberg) * The Twist: Instead of bursting balloons, find the minimum number of meeting rooms needed for a set of intervals. * **How to Solve:** This is the “Dual” of the balloon problem. Instead of finding how many intervals one arrow can hit, you find the maximum number of intervals that overlap at any single point in time. You would use a **Min-Heap** to track end times of meetings currently in rooms.

2. Variation: Task Scheduler / Cooling Time (Meta) * The Twist: You have tasks with a “cooling period” between same tasks. * **How to Solve:** This uses a Greedy approach with a **Frequency Map**. You prioritize the most frequent tasks to fill the schedule, similar to how we prioritize the earliest-ending balloons to save arrows.

3. Variation: Video Clipping/Buffering (Bloomberg) * The Twist: Given several video clips [start, end], what is the minimum number of clips needed to cover a whole 24-hour stream? * **How to Solve:** Sort by start time. In each step, pick the clip that starts within your current covered range but reaches the **furthest** into the future. This is another Greedy interval problem.

Employee Free Time

This is a classic “Hard” Leetcode problem. An L5/L6 engineer at Google would recognize this immediately as an **Interval Merging** problem.

The key insight is that we don’t care which employee is busy; we only care about the **global busy timeline**. If we merge all busy intervals from all employees into a single sorted timeline, the “gaps” between those merged busy blocks represent the times when **everyone** is free.

1. Problem Explanation

The Goal: You are given a list of employees. Each employee has a list of `[start, end]` intervals representing when they are busy. You need to find the free time intervals that are common to **all** employees.

The “Busy” vs. “Free” Intuition: Imagine a physical calendar. You take the busy schedules of every employee and highlight them with a marker on a single master calendar. The white spaces left on that calendar are the “Employee Free Time.”

Why it’s Hard: 1. The input is nested: A list of lists of intervals. 2. Intervals can overlap in messy ways (e.g., Employee A is busy 1-3, Employee B is busy 2-4). 3. We only want free time that is *finite* (don’t include time before the first meeting or after the last meeting).

2. Solution Explanation

The most efficient way to solve this is to flatten all schedules into one list, sort them by start time, and merge them.

Step 1: Flatten and Sort Collect every single `[start, end]` interval from every employee into one giant list. Sort this list by the **start** time.

Step 2: Merge Overlapping Intervals We keep track of the “latest end time” we have seen so far (let’s call it `temp_end`). * If the next interval starts *before or at* `temp_end`, it overlaps. We update `temp_end` to be the maximum of the current `temp_end` and the new interval’s end. * If the next interval starts *after* `temp_end`, we found a gap! The gap is from `temp_end` to the **start** of this new interval.

Visualization Walkthrough: Input: Employee 1: `[[1,3], [6,7]]`, Employee 2: `[[2,4]]`, Employee 3: `[[2,3], [9,12]]`

=====

STEP 1: FLATTEN AND SORT ALL BUSY TIMES

```

=====
Raw list: [[1,3], [6,7], [2,4], [2,3], [9,12]]

Sorted by Start Time:
1. [1, 3] (from Emp 1)
2. [2, 3] (from Emp 3)
3. [2, 4] (from Emp 2)
4. [6, 7] (from Emp 1)
5. [9, 12] (from Emp 3)

=====
STEP 2: MERGE AND FIND GAPS (ASCII DIAGRAM)
=====

Timeline: 1  2  3  4  5  6  7  8  9  10 11 12
          |--|--|--|--|--|--|--|--|--|---|---|---|

Interval 1: [1, 3]
Current Busy Block: [1, 3]
temp_end = 3

-----

Interval 2: [2, 3]
Starts at 2. Is 2 <= temp_end (3)? YES.
Overlap!
New temp_end = max(3, 3) = 3.
Current Busy Block: [1, 3]

-----

Interval 3: [2, 4]
Starts at 2. Is 2 <= temp_end (3)? YES.
Overlap!
New temp_end = max(3, 4) = 4.
Current Busy Block: [1, 4]

-----

Interval 4: [6, 7]
Starts at 6. Is 6 <= temp_end (4)? NO!
*** GAP DETECTED ***
Gap = [temp_end, current_start] = [4, 6]
Add [4, 6] to Result.

New temp_end = 7.
Current Busy Block: [6, 7]

-----

```



```
Interval 5: [9, 12]
Starts at 9. Is 9 <= temp_end (7)? NO!
*** GAP DETECTED ***
Gap = [temp_end, current_start] = [7, 9]
Add [7, 9] to Result.
```

```
New temp_end = 12.
Current Busy Block: [9, 12]
```

```
=====
FINAL RESULT: [[4, 6], [7, 9]]
=====
```

3. Time and Space Complexity Analysis

Let N be the total number of intervals across all employees.

TIME COMPLEXITY DERIVATION: $O(N \log N)$

- ```

```
1. Flattening:  
We touch every interval once. --->  $O(N)$
  2. Sorting:  
Sorting  $N$  intervals by start time. --->  $O(N * \log N)$
  3. Merging (Single Pass):  
We iterate through the sorted list once. --->  $O(N)$

Total Time =  $N + (N * \log N) + N$   
The  $\log N$  term is the bottleneck (dominant).  
Final Time Complexity:  $O(N * \log N)$

SPACE COMPLEXITY DERIVATION:  $O(N)$

- ```
-----
```
- What are we storing?
1. The flattened list of all intervals. ---> $O(N)$
 2. The output list of free intervals.
In the worst case (no overlaps), we could
have up to N free intervals. ---> $O(N)$

Total Space Complexity = $O(N)$

4. Solution Code

Python Solution (Flatten & Sort)

```
class Interval:
    def __init__(self, start: int = None, end: int = None):
        self.start = start
        self.end = end

def employeeFreeTime(schedule):
    # Step 1: Flatten all intervals into one list
    all_intervals = []
    for emp in schedule:
        for interval in emp:
            all_intervals.append(interval)

    # Step 2: Sort intervals by start time
    all_intervals.sort(key=lambda x: x.start)

    result = []
    if not all_intervals:
        return result

    # Step 3: Iterate and find gaps
    # Initialize temp_end with the end of the very first interval
    temp_end = all_intervals[0].end

    for i in range(1, len(all_intervals)):
        current = all_intervals[i]

        # If the current interval starts after our merged busy block ends
        if current.start > temp_end:
            # We found a gap between the end of the last busy block
            # and the start of this one.
            result.append(Interval(temp_end, current.start))

        # Always update temp_end to be the furthest point reached so far
        temp_end = max(temp_end, current.end)

    return result
```

JavaScript Solution (Flatten & Sort)

```
/**
 * Definition for an interval.
 * function Interval(start, end) {
 *   this.start = start;
```

```

    * this.end = end;
    * }
    */

/**
 * @param {Interval[][]} schedule
 * @return {Interval[]}
 */
var employeeFreeTime = function(schedule) {
    // Step 1: Flatten the nested arrays
    const allIntervals = [];
    for (let emp of schedule) {
        for (let interval of emp) {
            allIntervals.push(interval);
        }
    }

    // Step 2: Sort by start time
    allIntervals.sort((a, b) => a.start - b.start);

    const result = [];
    if (allIntervals.length === 0) return result;

    // Step 3: Merge and find gaps
    let tempEnd = allIntervals[0].end;

    for (let i = 1; i < allIntervals.length; i++) {
        let current = allIntervals[i];

        // If current start is greater than the end of the previous busy block
        if (current.start > tempEnd) {
            // Gap found! Add to result
            result.push({ start: tempEnd, end: current.start });
        }

        // Update tempEnd to the furthest point reached
        tempEnd = Math.max(tempEnd, current.end);
    }

    return result;
};

```

Note 1: Terminology and Techniques

- **Interval Merging:** A technique used to combine overlapping ranges. It is widely used in calendar apps and resource allocation systems.
 - **Sweep Line Algorithm:** While we used a simple sort-and-merge, this is a simplified version of a “Sweep Line.” We imagine a vertical line moving from left to right across time, processing “events” (the start and end of meetings).
 - **Heap / Priority Queue (Optimization):** An L6 might mention that if the employee schedules are *already sorted* (which they often are in LeetCode), we can use a **Min-Heap** to merge them in $O(N \log K)$ time where K is the number of employees, instead of $O(N \log N)$. This is better if there are thousands of intervals but only a few employees.
-

Note 2: Real-World Interview Variations

1. **Variation: Room Scheduling (Google)** * **The Problem:** Given a list of meetings, what is the minimum number of conference rooms required? * **How to solve:** This uses a Min-Heap. You sort by start time. For each meeting, check if the room that finishes earliest (heap top) is free. If not, you need a new room. If yes, you reuse the room and update its end time.
2. **Variation: Availability Picker (Meta)** * **The Problem:** Find a 30-minute slot where at least 3 out of 5 people are free. * **How to solve:** This is a “Coverage” problem. You mark every start as +1 and every end as -1. Sort these events. As you sweep through time, keep a running sum of people busy. Whenever $\text{Total_Employees} - \text{Running_Sum} \geq 3$, that time interval is a candidate.
3. **Variation: Bandwidth Usage (Bloomberg)** * **The Problem:** Each interval has a “weight” (bandwidth used). Find the time period with the maximum total bandwidth usage. * **How to solve:** Similar to the availability picker. Treat start as +weight and end as -weight. Sort by time. The peak of your running sum is the maximum bandwidth used. This is common in finance for tracking high-frequency trading volume peaks.

732. My Calendar III

Solving “My Calendar III” at a Google L5 (Senior) or L6 (Staff) level requires moving past simple interval checks and recognizing the problem as a **continuous coordinate system** challenge.

A senior engineer doesn’t just look for “overlaps”; they look for the **maximum density** of events at any single point in time. This is a classic “Sweep Line” problem.

1. Problem Explanation

The Goal: You need to build a calendar that allows you to book events. Every time you book an event `[start, end)`, you need to return the maximum number of overlapping events (the “K-booking”) currently in the system.

Key Rule: The interval is `[start, end)`, meaning the event includes the `start` time but *excludes* the `end` time. If one event ends at 10 and another starts at 10, they do **not** overlap.

Why it’s hard: As you add more events, the “K-booking” can change anywhere. We don’t just care about the new event; we care about the global peak of overlaps across the entire timeline.

2. Solution Explanation: The “Difference Array” / “Sweep Line”

Instead of thinking about intervals as blocks, think of them as **events that change the “load” on a timeline**.

- When an event **starts**, the number of active bookings increases by 1.
- When an event **ends**, the number of active bookings decreases by 1.

By tracking these “ups” and “downs” at specific timestamps, we can “sweep” across the timeline from left to right and calculate the active bookings at any moment.

ASCII Visualization of the Sweep Line Let’s say we book these three events in order: 1. `[10, 20)` 2. `[50, 60)` 3. `[10, 40)`

Step 1: Book `[10, 20)` * At time 10: +1 booking. * At time 20: -1 booking.
* Timeline: (10: +1), (20: -1) * Sweep: Start at 0... at 10 (count=1)... at 20 (count=0). Max = 1.

Step 2: Book `[50, 60)` * At time 50: +1. * At time 60: -1. * Timeline: (10: +1), (20: -1), (50: +1), (60: -1) * Sweep: Max = 1.

Step 3: Book `[10, 40)` * At time 10: +1. * At time 40: -1. * Timeline (Sorted): (10: +2), (20: -1), (40: -1), (50: +1), (60: -1)

The Final Sweep Walkthrough:

Time	Change	Current Active	Notes
10	+2	2	Two events started!
20	-1	1	One event ended.
40	-1	0	The second event ended.
50	+1	1	Another event starts.

60 | -1 | 0 | All clear.

Global Maximum Active Bookings = 2

L5/L6 Engineering Insight: In an interview, a top candidate will mention that a simple sorted map (like `TreeMap` in Java or `SortedDict` in Python) is the most straightforward way to keep these timestamps ordered. However, if the number of events is massive, they might suggest a **Segment Tree** for $O(\log N)$ updates, though the Sweep Line is usually the expected optimal balance for this specific LeetCode constraints.

3. Time and Space Complexity Analysis

Let N be the number of events booked so far.

TIME COMPLEXITY: $O(N * N)$

Each `book()` call:

1. Inserting into a sorted map: $O(N)$ or $O(\log N)$ depending on implementation.
2. Sweeping through the map: We must visit every timestamp currently stored. There are at most $2 * N$ timestamps. This takes $O(N)$ time.

Total for N bookings: $O(N * N)$

SPACE COMPLEXITY: $O(N)$

We store 2 timestamps (start and end) for every event in our map.

Total unique timestamps $\leq 2 * N$.

Total Space = $O(N)$

4. Solution Code

Python Implementation (Using `SortedDict`) *Note: In a standard interview, if `SortedDict` isn't available, you'd use a regular `dict` and `sorted(map.keys())`.*

```
from bisect import insort
```

```
class MyCalendarThree:
    def __init__(self):
```

```

# We store the "delta" (change) at each timestamp
self.changes = {}
# We keep track of sorted times to avoid O(N log N) sorting every time
self.sorted_times = []

def book(self, start: int, end: int) -> int:
    # Increment the count at start time
    self.changes[start] = self.changes.get(start, 0) + 1
    if start not in self.sorted_times:
        insert(self.sorted_times, start)

    # Decrement the count at end time
    self.changes[end] = self.changes.get(end, 0) - 1
    if end not in self.sorted_times:
        insert(self.sorted_times, end)

    max_bookings = 0
    current_active = 0

    # Sweep through the timestamps in chronological order
    for t in self.sorted_times:
        current_active += self.changes[t]
        if current_active > max_bookings:
            max_bookings = current_active

    return max_bookings

```

JavaScript Implementation

```

class MyCalendarThree {
    constructor() {
        // Map to store time -> delta change
        this.changes = new Map();
    }

    /**
     * @param {number} start
     * @param {number} end
     * @return {number}
     */
    book(start, end) {
        // Mark the start (+1) and end (-1)
        this.changes.set(start, (this.changes.get(start) || 0) + 1);
        this.changes.set(end, (this.changes.get(end) || 0) - 1);

        // Get all unique timestamps and sort them

```

```

const sortedTimes = Array.from(this.changes.keys()).sort((a, b) => a - b);

let maxBookings = 0;
let currentActive = 0;

// Sweep Line: Calculate running sum of active bookings
for (let t of sortedTimes) {
  currentActive += this.changes.get(t);
  // Track the highest value seen during the sweep
  if (currentActive > maxBookings) {
    maxBookings = currentActive;
  }
}

return maxBookings;
}
}

```

Note 1: Terms & Techniques

- **Sweep Line Algorithm:** A technique where you treat a 2D problem (intervals on a line) as a series of discrete events. You “sweep” a vertical line across the axis and update your state only when you hit an event.
 - **Difference Array:** The concept of only storing the *change* at the boundary of an interval rather than filling in every number inside the interval. This saves massive amounts of space.
-

Note 2: Real-World Interview Variations

1. **Meeting Room II (Google / Meta / Bloomberg) * The Problem:** Given a list of meeting times, what is the minimum number of conference rooms required to hold all meetings? * **The Connection:** This is the **exact same** problem. The number of conference rooms required is equal to the maximum number of overlapping intervals at any point. You would use the same Sweep Line logic.
2. **Server Load Monitoring (Bloomberg) * The Problem:** You have logs of when users log in and out. Find the peak number of concurrent users. * **The Connection:** Each login is a **start** (+1) and each logout is an **end** (-1). A Staff Engineer would suggest this approach because it handles millions of log entries efficiently by only processing the “events” (logins/logouts).
3. **Advertising Impressions (Google) * The Problem:** An ad runs from time X to Y. You have millions of ads. Find the moment in time when the most

ads were displayed simultaneously. * **The Connection:** Again, the Sweep Line. However, for “millions of ads,” the $O(N^2)$ total time might be too slow. A Google engineer would then discuss using a **Segment Tree with Lazy Propagation** to reduce the `book()` operation to $O(\log N)$.

Amount of New Area Painted Each Day

This is a sophisticated problem often used to distinguish L4 (Mid) from L5 (Senior) candidates at Google. A junior engineer might try a brute-force approach that fails on performance, while an L5/L6 engineer will focus on **path compression** and **interval management** to ensure the solution scales to massive inputs.

1. Problem Explanation

The Goal: You have a long fence starting at position 0. For several days, you are given a range `[start, end]` to paint. However, you **only** count the area that has **not been painted before**. You need to return an array where each index represents how much *new* area was painted on that specific day.

The Constraints: * A range `[1, 4]` covers 3 units of area (indices 1, 2, and 3). * If Day 1 paints `[1, 4]`, and Day 2 paints `[2, 6]`, Day 2 only gets credit for `[4, 5]` and `[5, 6]` because `[2, 4]` was already covered.

Example: * Day 0: `[1, 4]` -> New area: 3 (units 1, 2, 3) * Day 1: `[2, 5]` -> New area: 1 (unit 4, since 2 and 3 are already painted) * Day 2: `[8, 10]` -> New area: 2 (units 8, 9)

2. Solution Explanation

The Intuition (The “Why”): The naive way is to keep a Boolean array `isPainted`. For each day, you loop from `start` to `end`, check if it’s painted, if not, paint it and increment a counter. **Problem:** If you paint the same range `[0, 50000]` every day for 50,000 days, you perform 2.5 billion operations. This will “Time Limit Exceeded” (TLE).

The Senior Solution: Jump Pointers (Path Compression) We need a way to **skip** over segments that are already painted. Instead of just marking a spot as “painted,” we store a value that tells us: “If you land here, jump to the next unpainted spot.”

The Algorithm (The “How”): 1. Initialize an array `jump` (or a Map) where `jump[i]` tells you the next available spot to check. Initially, every spot points to itself (conceptually). 2. For each day’s range `[start, end]`: * Start a pointer `curr` at `start`. * While `curr < end`: * If `curr` has been painted, it will have a

jump[curr] value. Jump to it: `next_pos = max(curr + 1, jump[curr]).` *
If curr is NOT painted, increment today's count, paint it, and set `jump[curr]`
= end. * **Crucially:** Update `jump[curr]` as you move to point to the furthest
end seen so far. This is the "Jump" part.

ASCII Visualization Walkthrough: Input: `paint = [[1, 4], [2, 5]]`

[INITIAL STATE]

Fence: 0 1 2 3 4 5 6

Jump: [0,0,0,0,0,0,0] (Empty)

[DAY 0: [1, 4]]

Start curr = 1.

1. Is `jump[1]` set? No.
 - Count = 1 (painted unit 1).
 - Set `jump[1] = 4`.
 - curr moves to 2.
2. Is `jump[2]` set? No.
 - Count = 2 (painted unit 2).
 - Set `jump[2] = 4`.
 - curr moves to 3.
3. Is `jump[3]` set? No.
 - Count = 3 (painted unit 3).
 - Set `jump[3] = 4`.
 - curr moves to 4.

Day 0 Result: 3

Jump Array: [0, 4, 4, 4, 0, 0, 0]

[DAY 1: [2, 5]]

Start curr = 2.

1. Is `jump[2]` set? YES (it is 4).
 - We know area up to 4 is painted.
 - We record `old_curr = 2`.
 - curr JUMPS to 4.
 - update `jump[old_curr]` to 5 (path compression for future jumps).
2. curr is now 4. Is `jump[4]` set? No.
 - Count = 1 (painted unit 4).
 - Set `jump[4] = 5`.
 - curr moves to 5.

Loop ends because `curr (5) == end (5)`.

Day 1 Result: 1

Jump Array: [0, 4, 5, 4, 5, 0, 0]

3. Time and Space Complexity Analysis

TIME COMPLEXITY: $O(N + M)$

N = Number of days, M = Maximum coordinate (e.g., 50,000)

Even though we have a nested while-loop, each coordinate unit (1, 2, 3...) is "painted" exactly once.
Once a unit is painted, the 'jump' pointers ensure we leap over it in $O(1)$ or near $O(1)$ time in the future.
This is amortized linear time.

[Performance vs. Brute Force]

Brute Force: $O(N * M)$ -> $50,000 * 50,000 = 2.5$ Billion ops (FAIL)

Jump Logic: $O(N + M)$ -> $50,000 + 50,000 = 100,000$ ops (PASS)

SPACE COMPLEXITY: $O(M)$

We need an array to store the 'jump' pointers for the entire length of the fence.

[Memory Usage]

Fence Length (M): |--- 50,000 integers ---|

Result Array (N): |--- 50,000 integers ---|

Total Space = $O(M)$

4. Solution Code

Python Snippet

```
def amountPainted(paint):
    # max_pos is determined by the constraints (usually 50,001)
    # jump[i] stores the furthest 'end' we've reached starting from i
    jump = [0] * 50001
    result = []

    for start, end in paint:
        new_area = 0
        curr = start

        while curr < end:
```

```

        # If jump[curr] is 0, it means this unit is unpainted
        if jump[curr] == 0:
            jump[curr] = end
            new_area += 1
            curr += 1
        else:
            # If already painted, find the next possible unpainted spot
            # We also update jump[curr] to 'end' to compress the path
            next_pos = max(curr + 1, jump[curr])
            jump[curr] = max(jump[curr], end)
            curr = next_pos

    result.append(new_area)

    return result

```

JavaScript Snippet

```

/**
 * L5 Approach: Using Jump Pointers for Amortized Efficiency
 */
function amountPainted(paint) {
    const jump = new Int32Array(50001);
    const result = [];

    for (const [start, end] of paint) {
        let newArea = 0;
        let curr = start;

        while (curr < end) {
            if (jump[curr] === 0) {
                // Not painted: mark it and increment
                jump[curr] = end;
                newArea++;
                curr++;
            } else {
                // Already painted: jump ahead
                const nextPos = Math.max(curr + 1, jump[curr]);
                // Update the jump pointer (Path Compression)
                jump[curr] = Math.max(jump[curr], end);
                curr = nextPos;
            }
        }
        result.push(newArea);
    }
}

```

```
    return result;
}
```

Note 1: Terminology Addendum

Path Compression: * **What it is:** An optimization technique where, while looking for a value, you update the shortcuts you found along the way so that the *next* search is even faster. * **Why it helps:** It prevents the algorithm from doing the same work twice. In this problem, it prevents us from re-scanning intervals we already know are painted. * **Application:** We use it here to update the `jump` array to point to the latest `end` point.

Note 2: Real-World Interview Variations

Variation 1: “Merge Intervals” (The Foundation) * *How it’s asked:* “Given a list of intervals, merge all overlapping ones.” * *L5 Solve:* Sort by start time, then iterate. This is the simpler version of the “Painted Area” problem because it doesn’t require calculating *new* area on specific days.

Variation 2: “Range Module” (Hard - Google/Meta) * *How it’s asked:* “Design a data structure that tracks ranges and can answer if a range is fully covered or remove a range.” * *L5 Solve:* A senior engineer would likely use a **Segment Tree** or a **TreeMap** (in Java/C++) to maintain a sorted set of non-overlapping intervals. When a new range is added, you “swallow” all internal intervals and merge the boundaries.

Variation 3: “Disk Space Allocation” (System Design Context) * *How it’s asked:* “How does an OS track which blocks on a 1TB drive are free vs. used?” * *L5 Solve:* This is exactly the “Amount of New Area Painted” problem. Using a “Bitmap” is the naive approach (like our Boolean array). Modern file systems use **Extents** (ranges of blocks) and **B-Trees** to jump over allocated chunks of data efficiently, similar to our `jump` pointer logic.

715. Range Module

This is a high-difficulty problem (LeetCode Hard). A Google L5/L6 engineer views this as a **Symptom of Data Management**. The core challenge isn’t just adding numbers, but managing “State” efficiently. In a production environment, this is essentially how a **Memory Allocator** or a **File System** tracks which blocks of disk/RAM are in use and which are free.

1. Problem Explanation

The Premise: You need to design a system that tracks ranges of numbers (half-open intervals like `[left, right)`). You must support: 1. `addRange(left, right)`: Add the range to your tracking. If it overlaps with existing ranges, they merge. 2. `queryRange(left, right)`: Check if the **entire** range `[left, right)` is currently being tracked. 3. `removeRange(left, right)`: Stop tracking any part of the ranges that fall within `[left, right)`.

The Intuition (The “Why”): Think of this as a **Timeline of Ownership**. - `addRange` is like buying a plot of land. If you buy land next to land you already own, it becomes one big farm. - `removeRange` is like the government taking land back for a highway. It might cut your big farm into two smaller pieces or delete a piece entirely. - `queryRange` is asking: “Do I own every single inch of ground from point A to point B?”

Crucial Detail: The ranges are “half-open” `[10, 20)`. This means it includes 10, 11, ... 19. It does **not** include 20. This makes merging easier because `[10, 20)` and `[20, 30)` meet perfectly at 20.

2. Solution Explanation

A Senior Engineer knows that the most efficient way to manage sorted, non-overlapping intervals is a **Sorted Map** (in Java/C++) or a **Sorted List of Pairs** (in Python/JS).

The Strategy: We will maintain a sorted list of disjoint (non-overlapping) intervals. - **Add:** Find all intervals that touch the new range. Merge them into one giant interval. - **Remove:** Find all intervals that touch the removal range. Delete them, but “trim” the edges if they extend outside the removal zone. - **Query:** Find the one interval that *should* contain our query. Check if its boundaries fully cover it.

Detailed ASCII Walkthrough Let’s look at a complex sequence: `add(10, 20)`, then `add(15, 25)`, then `remove(18, 22)`.

Step 1: `addRange(10, 20)` The module is empty. Current State: `[[10, 20]]`

Step 2: `addRange(15, 25)` We check where 15 and 25 fall. 15 is inside `[10, 20]`. 25 is outside. We merge them: `min(10, 15)` to `max(20, 25)`.

```
Existing:  [10 ----- 20]
New:       [15 ----- 25]
Result:    [10 ----- 25]

Current State: [[10, 25]]
```

Step 3: removeRange(18, 22) This is the hardest part. 18 to 22 is right in the middle of our only interval. We must “split” the existing interval.

```
Existing:  [10 ----- 25]
Remove:           [18 ----- 22]
              (Keep)      (Cut)      (Keep)
Result:    [10 ----- 18]      [22 ----- 25]
Current State: [[10, 18], [22, 25]]
```

Step 4: queryRange(15, 23) Is the whole block [15, 23] tracked? Looking at our state: We have [10, 18] and [22, 25]. There is a gap between 18 and 22!

```
Query:           [15 ----- 23]
State:    [10 -- 18]      [22 -- 25]
Gap found! -----> [18-22] is missing.
Result: FALSE
```

3. Time and Space Complexity Analysis

Time Complexity: We use Binary Search (Bisect) to find the insertion points.

Visual Derivation of Time Complexity:

N = Number of existing intervals.

1. Find overlap start/end: Binary Search takes $O(\log N)$.
2. Modify list:
 - In Python/JS, deleting or inserting into a list takes $O(N)$.
 - In a Balanced BST (like C++ std::map), this is $O(\log N)$.

Total Time per operation:

Python/JS: $O(N)$ worst case for add/remove. $O(\log N)$ for query.

C++/Java: $O(\log N)$ for all.

Space Complexity:

Visual Derivation of Space Complexity:

We store intervals as pairs in a list.

[[s1, e1], [s2, e2], ... [sn, en]]

If we have N disjoint intervals:

Memory used = $2 * N$ units.

Total Space: $O(N)$

4. Solution Code

I'll provide the **Sorted List + Bisect** approach. This is the most practical to implement in an interview environment while remaining highly performant.

Python Implementation

```
import bisect

class RangeModule:
    def __init__(self):
        # We store intervals as a flat list: [start1, end1, start2, end2...]
        # This allows us to use bisect on a single list.
        self.ranges = []

    def addRange(self, left: int, right: int) -> None:
        # Find where 'left' and 'right' would fit in our sorted points
        i = bisect.bisect_left(self.ranges, left)
        j = bisect.bisect_right(self.ranges, right)

        new_interval = []
        # If 'i' is even, 'left' is outside an existing range. It starts a new one.
        if i % 2 == 0:
            new_interval.append(left)
        # If 'i' is odd, 'left' is inside a range. We keep the existing start.
        else:
            new_interval.append(self.ranges[i-1])
            i -= 1 # Adjust index for replacement

        # Same logic for the 'right' side
        if j % 2 == 0:
            new_interval.append(right)
        else:
            new_interval.append(self.ranges[j])
            j += 1 # Adjust index for replacement

        # Replace everything between i and j with our new merged boundaries
        self.ranges[i:j] = new_interval

    def queryRange(self, left: int, right: int) -> bool:
        # Find where 'left' and 'right' would be inserted
        i = bisect.bisect_right(self.ranges, left)
        j = bisect.bisect_left(self.ranges, right)
        # For a query to be true, BOTH must fall inside the SAME existing range.
```



```

    # This means they must both be at the same odd index.
    return i == j and i % 2 == 1

def removeRange(self, left: int, right: int) -> None:
    # Removal is the 'inverse' of addition
    i = bisect.bisect_left(self.ranges, left)
    j = bisect.bisect_right(self.ranges, right)

    new_interval = []
    if i % 2 == 1:
        new_interval.append(left)
    if j % 2 == 1:
        new_interval.append(right)

    self.ranges[i:j] = new_interval

```

JavaScript Implementation

```

class RangeModule {
  constructor() {
    this.ranges = [];
  }

  // Helper for binary search since JS doesn't have a built-in 'bisect'
  bisect(val) {
    let low = 0, high = this.ranges.length;
    while (low < high) {
      let mid = Math.floor((low + high) / 2);
      if (this.ranges[mid] < val) low = mid + 1;
      else high = mid;
    }
    return low;
  }

  addRange(left, right) {
    let i = this.bisect(left);
    let j = this.bisect(right);

    // If 'right' is exactly at a boundary, adjust 'j'
    while (j < this.ranges.length && this.ranges[j] <= right) j++;

    let newRange = [];
    if (i % 2 === 0) newRange.push(left);
    if (j % 2 === 0) newRange.push(right);

    this.ranges.splice(i, j - i, ...newRange);
  }
}

```

```

    }

    queryRange(left, right) {
        let i = this.bisect(left + 1); // Use left+1 to check if inside
        let j = this.bisect(right);
        // If i is odd, left is in a range. If i == j, no gaps exist.
        return i % 2 === 1 && i === j;
    }

    removeRange(left, right) {
        let i = this.bisect(left);
        let j = this.bisect(right);
        while (j < this.ranges.length && this.ranges[j] < right) j++;

        let newRange = [];
        if (i % 2 === 1) newRange.push(left);
        if (j % 2 === 1) newRange.push(right);

        this.ranges.splice(i, j - i, ...newRange);
    }
}

```

Note 1: Terms and Techniques

- **Flat Sorted List (The Boundary Point Method):** Instead of storing `[[start, end]]`, we store `[start, end, start, end]`. All even indices (0, 2...) are starts, all odd indices (1, 3...) are ends. This is a very clever way to simplify the logic of “am I inside or outside a range?”
- **Pigeonhole/Parity Logic:** Using `% 2` to determine state is a common “Senior” trick. If you are past an even number of points, you are currently “outside” an owned range. If you are past an odd number, you are “inside”.

Note 2: Real-World Interview Variations

1. Bloomberg: “The Bloomberg Terminal Ticker History” * The Twist: You have stock price data arriving in ranges. Some data is corrected later (removal). You need to answer: “Do we have a complete, gapless history of the price for ‘AAPL’ from 9:00 AM to 4:00 PM?” * **L5 Solution:** Use a `Map<Ticker, RangeModule>`. Each ticker gets its own `RangeModule` to track data availability.

2. Google: “Cloud Resource Allocator” * The Twist: You have a cluster of 10,000 CPUs. Users request a range of CPUs (e.g., “I need CPUs 500 to 1000”). You must track which are busy and which are free. * **L5 Solution:**

This is the `removeRange` (allocating CPUs) and `addRange` (releasing CPUs) logic. The `queryRange` is used to check if a requested block is fully available.

3. Meta: “Video Privacy Settings” * The Twist: A video is 60 minutes long. A user marks certain segments as “Private”. You need to know if a specific 5-minute clip is entirely public (query). * **L5 Solution:** Treat the “Private” timestamps as `addRange`. The clip is “safe” to show if `queryRange` on the Private ranges returns `FALSE` for every single second of that clip.

1943. Describe the Painting

To a senior engineer, this problem isn’t about “painting”; it’s about **Interval Management** and **Prefix Sums**. When you see multiple overlapping intervals where you need to track a cumulative value (the color sum), your mind should immediately jump to the **Difference Array** (or Sweep Line) technique.

Here is the engineering breakdown of LeetCode 1943.

1. Problem Explanation

The Goal: You have a long canvas (a number line). Different segments of this canvas are painted with colors. Each “color” is actually an integer value. If segments overlap, their values are added together. You need to return a list of non-overlapping segments `[start, end, total_color]`.

The Crucial Rule: If two adjacent segments have the same total color but were formed by different sets of original paint strokes, you **cannot** merge them. You must split segments whenever a paint stroke starts or ends.

Visualizing the Problem: `segments = [[1, 4, 5], [4, 7, 7], [1, 7, 9]]`

Strokes:

```
1 (1 to 4, val 5):   [ 5 5 5 ]
2 (4 to 7, val 7):           [ 7 7 7 ]
3 (1 to 7, val 9):   [ 9 9 9   9 9 9 9 ]
```

```
Timeline:           1   2   3   4   5   6   7
Combined Sums:      |  14  14  |  16  16  |
                    +-----+-----+
```

Result: `[[1, 4, 14], [4, 7, 16]]`

2. Solution Explanation

The Bottleneck: If the canvas is 100,000 units long, you can't just create an array and update every index for every stroke. That would be $O(N * \text{Number of Strokes})$, which is too slow.

The “Difference Array” Strategy: Instead of painting every unit, we only record the **changes**. 1. When a stroke starts at **start** with value v , the total color increases by v . 2. When a stroke ends at **end** with value v , the total color decreases by v .

We use a Hash Map (or a sorted array) to store these changes at specific coordinates.

The “Split” Requirement: The problem says we must split segments if the set of colors changes, even if the sum stays the same. *Example:* Stroke A ends at 4, Stroke B starts at 4. Even if `sum(A) == sum(B)`, index 4 is a “boundary.” *Solution:* By only iterating through the **sorted coordinates** where strokes start or end, we naturally respect every boundary.

ASCII Diagram Walkthrough segments = [[1, 4, 5], [1, 4, 7], [4, 7, 12]]

Step 1: Map the Changes

Stroke [1, 4, 5]: Point 1: +5, Point 4: -5
Stroke [1, 4, 7]: Point 1: +7, Point 4: -7
Stroke [4, 7, 12]: Point 4: +12, Point 7: -12

Final Change Map:

Point 1: +12

Point 4: +0 (-5 -7 +12) <-- Notice the sum is 0, but 4 is still a boundary!

Point 7: -12

Step 2: Sweep Line (Prefix Sum) We sort the points: [1, 4, 7].

Point 1:

Current Color Sum = 0 + 12 = 12

Next Point is 4.

Segment found: [1, 4, 12]

Point 4:

Current Color Sum = 12 + 0 = 12

Next Point is 7.

Segment found: [4, 7, 12]

(We do NOT merge [1, 4] and [4, 7] because 4 was a transition point)

Point 7:

Current Color Sum = 12 - 12 = 0
End of canvas.

3. Time and Space Complexity Analysis

TIME COMPLEXITY DERIVATION: $O(N \log N)$
N = Number of paint segments (input size)
1. Building the Map: Iterate through N segments, update 2 points each. Cost: $O(N)$
2. Sorting Coordinates: There are at most $2N$ coordinates in the map. Sorting them: Cost: $O(N \log N)$
3. Sweeping: Iterate through the sorted unique coordinates once. Cost: $O(N)$
TOTAL TIME: $O(N) + O(N \log N) + O(N) \implies O(N \log N)$
SPACE COMPLEXITY DERIVATION: $O(N)$
1. Change Map (Dictionary/Hash Table): Stores at most $2N$ entries for the start and end points. Cost: $O(N)$
2. Sorted Keys List: Stores at most $2N$ coordinates. Cost: $O(N)$
TOTAL SPACE: $O(N)$

4. Solution Code

Python Snippet

```

def describePainting(segments):
    # Use a dictionary to store the change in color value at each point
    # Mapping: coordinate -> color_change
    mapping = {}

    for start, end, color in segments:
        mapping[start] = mapping.get(start, 0) + color
        mapping[end] = mapping.get(end, 0) - color

    # We must sort the coordinates to "sweep" from left to right
    sorted_points = sorted(mapping.keys())

    res = []
    current_sum = 0

    # Iterate through the points and build segments between neighbors
    for i in range(len(sorted_points) - 1):
        start_pt = sorted_points[i]
        end_pt = sorted_points[i+1]

        # Accumulate the changes to get the sum for the current segment
        current_sum += mapping[start_pt]

        # If current_sum is 0, it means this part of the canvas is unpainted
        if current_sum > 0:
            res.append([start_pt, end_pt, current_sum])

    return res

```

JavaScript Snippet

```

function describePainting(segments) {
    // Map to store net color change at each coordinate
    const mapping = new Map();

    for (const [start, end, color] of segments) {
        mapping.set(start, (mapping.get(start) || 0) + color);
        mapping.set(end, (mapping.get(end) || 0) - color);
    }

    // Get unique points and sort them numerically
    const sortedPoints = Array.from(mapping.keys()).sort((a, b) => a - b);

    const res = [];
    let currentSum = 0;

    // Build intervals based on adjacent sorted points

```

```

for (let i = 0; i < sortedPoints.length - 1; i++) {
    const start = sortedPoints[i];
    const end = sortedPoints[i + 1];

    // Update the running prefix sum
    currentSum += mapping.get(start);

    // Only add segments that have paint (sum > 0)
    if (currentSum > 0) {
        // Note: Use BigInt if color values could exceed 2^53 - 1,
        // but for standard LC problems, regular numbers suffice.
        res.push([start, end, currentSum]);
    }
}

return res;
}

```

Note 1: Terminology and Techniques

- **Difference Array / Sweep Line:** Instead of updating an entire range $[L, R]$, you mark $+Value$ at L and $-Value$ at $R+1$. Calculating the prefix sum of this array restores the original values. It turns an $O(RangeWidth)$ operation into $O(1)$.
- **Coordinate Compression:** This problem uses a variation of this. We don't care about the millions of points *between* 1 and 4; we only care about 1 and 4 themselves. By only processing "interesting" points, we save massive amounts of time and space.

Note 2: Real-World Interview Variations

1. Google: "Cloud Resource Usage Billing" * Prompt: A server has multiple jobs running. Job A uses 5GB RAM from 1pm to 4pm. Job B uses 10GB from 2pm to 6pm. Generate a report showing the total RAM usage in distinct time intervals. *** Solution:** Identical to LC 1943. Use a Sweep Line on the timestamps. The "color" is the RAM usage. You must split segments at every Job Start/End to ensure billing accuracy.

2. Meta: "Live Stream Viewer Count" * Prompt: Given a list of user login and logout times, find the maximum number of concurrent viewers and the time periods when that peak occurred. *** Solution:** Use the Difference Array. Increment at `login`, decrement at `logout`. As you sweep through, keep track of the `max_viewers`. Collect the segments where `current_sum == max_viewers`.

3. Bloomberg: “Stock Volume Spikes” * **Prompt:** Multiple trading algorithms are active at different times, each contributing a certain volume of trades per second. Identify the time windows where the combined volume exceeds a certain threshold. * **Solution:** Sweep Line again. The “threshold” check is done during the final loop where you build the segments. If `current_sum > threshold`, record that window.

1094. Car Pooling

At the L5/L6 level at Google, we look for solutions that move away from “simulation” and toward “analytical transformations.” A junior dev might try to simulate the car driving mile-by-mile, but a senior engineer recognizes this as a **Difference Array** problem.

In a real-world system, you don’t care about the empty road; you only care about the **events**—when people get in and when they get out.

1. Problem Explanation

The Core Question: You have a car with a fixed capacity. You are given a list of trips where each trip is `[num_passengers, start_location, end_location]`. Can you complete all trips without ever exceeding the car’s capacity?

The Crucial Rule: At any given location `X`, the number of passengers currently in the car must be less than or equal to `capacity`.

Visualizing the “Flow”: Imagine a car moving along a straight road (the `X`-axis).

Case A: Valid Car Pool Trips: `[[2, 1, 5], [3, 3, 7]]`, Capacity: 5

Location:	0	1	2	3	4	5	6	7
Trip 1:		[+2----->-2]						
Trip 2:			[+3----->-3]					

Passenger Count per Mile:
 Mile 1-2: 2 (OK, 2 ≤ 5)
 Mile 3-4: 5 (OK, 2+3 ≤ 5)
 Mile 5-6: 3 (OK, 5-2 = 3)
 Mile 7+: 0

Result: TRUE.

Case B: Invalid Car Pool Trips: `[[2, 1, 5], [3, 3, 7]]`, Capacity: 4


```
Location: 3    4    5
           |    |    |
Mile 3-4: 5 passengers.
Wait! Capacity is only 4.
Result: FALSE.
```

2. Solution Explanation

To solve this efficiently, we use a technique called **Difference Array (or Bucket Sort for Events)**.

Instead of tracking every mile, we only track the **changes** at specific locations.
* If n people get in at **start**, we do: `road[start] += n` * If n people get out at **end**, we do: `road[end] -= n`

The Logic: Once we mark all the “ins” and “outs,” we walk through the road from the beginning. We keep a running sum (a prefix sum). This running sum represents the exact number of people in the car at that specific location.

Walkthrough Visualization: Trips: `[[2, 1, 5], [3, 3, 7]]`, Capacity: 5

Step 1: Initialize an array of 1001 zeros (since locations are 0-1000).
Road: `[0, 0, 0, 0, 0, 0, 0, 0, ...]`

Step 2: Process Trip `[2, 1, 5]`
- At Mile 1, add 2 people: `Road[1] = 2`
- At Mile 5, remove 2 people: `Road[5] = -2`

Step 3: Process Trip `[3, 3, 7]`
- At Mile 3, add 3 people: `Road[3] = 3`
- At Mile 7, remove 3 people: `Road[7] = -3`

The Road Array (Events only):
Index: 0 1 2 3 4 5 6 7
Value: `[0, +2, 0, +3, 0, -2, 0, -3]`

Step 4: The Running Sum Scan
- Mile 0: Sum = 0 (OK)
- Mile 1: Sum = 0 + 2 = 2 (OK)
- Mile 2: Sum = 2 + 0 = 2 (OK)
- Mile 3: Sum = 2 + 3 = 5 (OK)
- Mile 4: Sum = 5 + 0 = 5 (OK)
- Mile 5: Sum = 5 - 2 = 3 (OK)
- Mile 6: Sum = 3 + 0 = 3 (OK)
- Mile 7: Sum = 3 - 3 = 0 (OK)

All sums were ≤ 5 . Return TRUE.

3. Time and Space Complexity Analysis

Let N be the number of trips and L be the maximum possible location (1000 in this problem).

=====

TIME COMPLEXITY: $O(N + L)$

=====

Derivation Diagram:

1. Process Trips:
 [Trip 1] $\rightarrow$ [Trip 2] $\rightarrow$... $\rightarrow$ [Trip N]
 For each trip, we do exactly 2 array updates.
 Cost = $O(N)$
2. Scan the Road:
 [Mile 0] $\rightarrow$ [Mile 1] $\rightarrow$... $\rightarrow$ [Mile 1000]
 We walk through the entire fixed-size road array.
 Cost = $O(L)$

Total Time = $O(N + L)$

=====

=====

SPACE COMPLEXITY: $O(L)$

=====

Derivation Diagram:

[Difference Array Storage]
The array size depends ONLY on the maximum location possible (1001 slots).

Regardless of how many trips (N) we have (10 or 10,000), the space required stays fixed at 1001.

Total Space = $O(L)$

=====

4. Solution Code

Python

```

def carPooling(trips, capacity):
    # The problem constraints say distance is 0 to 1000
    # We use 1001 to include the index 1000
    road = [0] * 1001

    # Step 1: Mark all entries and exits
    # For every trip, passengers increase at start and decrease at end
    for num_passengers, start, end in trips:
        road[start] += num_passengers
        road[end] -= num_passengers

    # Step 2: Traverse the road and calculate running sum
    current_passengers = 0
    for passengers_change in road:
        current_passengers += passengers_change

        # If at any point we exceed capacity, return False
        if current_passengers > capacity:
            return False

    return True

```

JavaScript

```

/**
 * L5/L6 Approach: Difference Array / Bucket Sort
 * This avoids sorting  $O(N \log N)$  by taking advantage of
 * the fixed range of locations.
 */
function carPooling(trips, capacity) {
    // Create an array to track changes at each location
    const road = new Array(1001).fill(0);

    // Step 1: Record passenger changes
    for (const [num, start, end] of trips) {
        road[start] += num;
        road[end] -= num;
    }

    // Step 2: Check running sum against capacity
    let currentCapacity = 0;
    for (let i = 0; i < road.length; i++) {
        currentCapacity += road[i];

        // At any "event" mile, if we exceed capacity, it's impossible
        if (currentCapacity > capacity) {

```

```

        return false;
    }
}

return true;
}

```

Note 1: Terms and Techniques

- **Difference Array:** A technique where you represent an array by the differences between consecutive elements. It is extremely powerful for “range update” problems. Instead of updating every index from **start** to **end** (which is slow), you only update the boundaries.
 - **Prefix Sum:** The cumulative sum of an array. In this problem, the prefix sum of the Difference Array restores the original “passenger count” at every mile.
 - **Bucket Sort (Conceptually):** Because we know the locations are limited to 1000, we use the array indices as “buckets,” which allows us to achieve linear time $O(N)$ instead of the $O(N \log N)$ required if we had to sort the trips.
-

Note 2: Real-World & Interview Variations

1. Google: “Corporate Flight Bookings” (LeetCode 1109) * **The Problem:** Given flight bookings (start seat, end seat, num seats), return the total seats booked for each flight. * **The Solution:** This is the **exact** same problem. Use a difference array to mark the start and end of bookings, then calculate the prefix sum to get the final seat counts for all flights.

2. Meta: “Meeting Rooms II” (Heap vs. Difference Array) * **The Problem:** Find the minimum number of conference rooms needed. * **The Solution:** If the time range is small (like 0-1000), use the **Difference Array** from this carpooling solution! If the time range is huge (like 0 to 1 billion), you can’t build a massive array. Instead, you extract all “start” and “end” times, sort them as events, and process them in order.

3. Bloomberg: “Video Buffering / Bandwidth Monitor” * **The Problem:** You have multiple video streams starting and ending at different times, each using some bandwidth. Does your total bandwidth ever exceed a limit? * **The Solution:** This is Car Pooling in a different skin. The “passengers” are “kilobytes per second” and the “locations” are “timestamps.” You track when streams start and stop. Use a **TreeMap** (in Java/C++) or a sorted list of events if the timestamps are high-precision (not integers).

1353. Maximum Number of Events That Can Be Attended

This is a classic “Greedy + Priority Queue” problem. A Google L5/L6 engineer views this through the lens of **Urgency Management**. In a high-scale environment, this is analogous to a CPU task scheduler or a logistics system where tasks have “Ready Times” and “Deadlines.”

The core intuition is simple: **Always attend the event that is going to expire the soonest.**

1. Problem Explanation

The Goal: You are given a list of events where each event is [startDay, endDay]. You can attend at most one event on any given day. You want to maximize the total number of events you attend.

The Constraints: * You can only attend an event if startDay <= currentDay <= endDay. * You can only pick one event per day.

The “Why” it’s hard: If you have two events starting on Day 1: Event A: [1, 2] Event B: [1, 5] Which should you pick? If you pick B on Day 1, you might lose the chance to attend A because A expires sooner. If you pick A on Day 1, you can still attend B on Day 2, 3, 4, or 5.

Senior Engineer’s realization: 1. We must process days chronologically (Day 1, then Day 2...). 2. On any day d, we first identify all events that have just become “available” (startDay == d). 3. From all available events, we pick the one with the **earliest endDay**. This is the “Greedy” choice.

2. Solution Explanation

We use a **Min-Heap (Priority Queue)** to keep track of the endDay of all events we could currently attend.

The Algorithm Steps:

1. **Sort** the events by their startDay.
2. Iterate through days from 1 to 100,000 (or the max end day).
3. **Add** to the Heap all events that start today.
4. **Remove** from the Heap all events that ended yesterday (they are now impossible to attend).
5. **Pick** the event from the Heap that ends the soonest and attend it.

Step-by-Step Visualization Input: events = [[1,2], [2,3], [3,4], [1,2]]

=====

INITIAL STATE

=====

Sorted Events: [[1,2], [1,2], [2,3], [3,4]]
Current Day: 1
Heap (End Days): []
Events Attended: 0

=====

DAY 1

=====

1. Available Events: [1,2] and [1,2] (both start on Day 1)
2. Add End Days to Heap:
Heap: [2, 2] (Two events ending on Day 2)
3. Cleanup: No events in heap ended before Day 1.
4. Attend Event: Pop 2 from heap.
Events Attended: 1
Heap: [2]

=====

DAY 2

=====

1. Available Events: [2,3] (starts on Day 2)
2. Add End Day to Heap:
Heap: [2, 3] (One ending Day 2, one ending Day 3)
3. Cleanup: None.
4. Attend Event: Pop 2 from heap (the most urgent one!).
Events Attended: 2
Heap: [3]

=====

DAY 3

=====

1. Available Events: [3,4] (starts on Day 3)
2. Add End Day to Heap:
Heap: [3, 4]
3. Cleanup: None.
4. Attend Event: Pop 3 from heap.
Events Attended: 3
Heap: [4]

=====

DAY 4

```

=====
1. Available Events: None.
2. Add to Heap: Nothing.
3. Cleanup: None.
4. Attend Event: Pop 4 from heap.
   Events Attended: 4
   Heap: []

```

FINAL RESULT: 4

3. Time and Space Complexity Analysis

Let N be the number of events and D be the range of days.

TIME COMPLEXITY DERIVATION: $O(N \log N + D \log N)$

```

1. Sorting Events:
   N events sorted by start day.          --->  $O(N \log N)$ 

2. Iterating through Days (D):
   We loop through each day.              --->  $O(D)$ 
   Inside the loop:
     - Push events into Heap:
       Each event pushed once.             --->  $O(N \log N)$  total
     - Pop expired events:
       Each event popped once.             --->  $O(N \log N)$  total
     - Pop one event to attend:
       D pops total.                       --->  $O(D \log N)$ 

```

Total Time = $O(N \log N) + O(D \log N)$
 (Usually, N and D are similar in scale, so $O(N \log N)$).

SPACE COMPLEXITY DERIVATION: $O(N)$

```

What are we storing?
1. Sorted list of events:                  --->  $O(N)$ 
2. Priority Queue (Heap):
   In the worst case, all events start on
   Day 1 and we store all  $N$  end days.     --->  $O(N)$ 

```

OVERALL SPACE: $O(N)$

4. Solution Code

Python Implementation (Using heapq)

```
import heapq

def maxEvents(events):
    # Step 1: Sort events by start day
    events.sort()

    max_attended = 0
    heap = [] # This will store endDays of available events
    event_idx = 0
    n = len(events)

    # We start from the earliest start day and move forward
    current_day = events[0][0]

    # Continue while there are events to add or events in the heap
    while event_idx < n or heap:
        # 1. If heap is empty, jump to the start day of the next event
        if not heap and event_idx < n:
            current_day = events[event_idx][0]

        # 2. Add all events starting today into the heap
        while event_idx < n and events[event_idx][0] <= current_day:
            heapq.heappush(heap, events[event_idx][1])
            event_idx += 1

        # 3. Remove events that ended before current_day
        while heap and heap[0] < current_day:
            heapq.heappop(heap)

        # 4. Attend the most urgent event (one with earliest endDay)
        if heap:
            heapq.heappop(heap)
            max_attended += 1
            current_day += 1 # We used 1 day to attend this event

    return max_attended
```

JavaScript Implementation (Using a simple Min-Heap)

```
/**
 * Note: JavaScript does not have a built-in Priority Queue.
 * In a real interview, you might have to implement a basic one
 * or discuss the logic as if one exists.
```



```

*/
var maxEvents = function(events) {
    // Sort by start day
    events.sort((a, b) => a[0] - b[0]);

    let heap = new MinHeap(); // Assume a MinHeap implementation exists
    let maxAttended = 0;
    let i = 0;
    let n = events.length;
    let day = 0;

    while (i < n || !heap.isEmpty()) {
        if (heap.isEmpty()) {
            day = events[i][0];
        }

        // Add events starting today
        while (i < n && events[i][0] <= day) {
            heap.push(events[i][1]);
            i++;
        }

        // Remove expired events
        while (!heap.isEmpty() && heap.peak() < day) {
            heap.pop();
        }

        // Attend the best event
        if (!heap.isEmpty()) {
            heap.pop();
            maxAttended++;
            day++;
        }
    }

    return maxAttended;
};

```

Note 1: Terminology & Techniques

- **Min-Heap (Priority Queue):** A tree-based data structure where the smallest (or largest) element is always at the top. It allows us to retrieve the “most urgent” event in $O(\log N)$ time.
- **Greedy Algorithm:** A strategy that makes the locally optimal choice

at each stage (picking the earliest deadline) with the hope of finding the global optimum. For this specific scheduling problem, Greedy is mathematically proven to be optimal.

Note 2: Real-World Interview Variations

1. Variation: Meeting Rooms III (Google) * The Twist: You have k rooms. You must attend events, but if a room isn't free, the event is delayed. You need to find which room held the most meetings. * **How to solve:** This combines the `endDay` heap from this problem with a second heap to track `availableRoomIds`. It tests your ability to manage multiple stateful priorities simultaneously.

2. Variation: Maximum Profit in Job Scheduling (Meta/Bloomberg)
* **The Twist:** Each event now has a `profit`. You want to maximize profit, not just the number of events. * **How to solve:** Greedy no longer works here because a long, low-profit event might block many short, high-profit ones. You must use **Dynamic Programming** combined with **Binary Search** to find the max profit for each time interval.

3. Variation: Task Scheduler (Meta) * The Twist: You have tasks with different types and a “cooling period” n . You must find the minimum time to finish all tasks. * **How to solve:** You use a Max-Heap based on the *frequency* of tasks. This is the inverse of our “End Day” logic—here, the most frequent task is the most “urgent” because it needs the most time to clear its cooling cycles.